

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import os
import pandas as pd
import numpy as np

# -----
# CONFIG
# -----
FOLDER_PATH = "/content/drive/MyDrive/VI Projects Dataset : OCA Blast"
```

```
# -----
# BACA SEMUA CSV
# -----
dfs = {}
for filename in os.listdir(FOLDER_PATH):
    if filename.endswith(".csv"):
        filepath = os.path.join(FOLDER_PATH, filename)
        df = pd.read_csv(filepath)
        dfs[filename] = df
        print(f"\n{'='*50}")
        print(f"FILE : {filename}")
        print(f"Shape : {df.shape}")
        print(f"Cols : {list(df.columns)}")
        print(df.head(3))
```

```
=====
FILE : Call.csv
Shape : (12749, 6)
Cols : ['transaction_id', 'user_id', 'is_charge', 'total_price', 'created_at', 'message_status']
      transaction_id user_id is_charge total_price \
0  6863b22a7ecd7c0012e51bd4  user_1      False      0
1  6864b619e3434a0019b1fb07  user_1      False      0
2  6864ff3f7ecd7c0012e54de7  user_1      False      0
```

```
      created_at message_status
0  0025-01-29 09:13:00 UTC      rna
1  0025-01-22 13:06:00 UTC      rna
2  0025-02-25 17:48:00 UTC      rna
```

```
=====
FILE : Email.csv
Shape : (40000, 6)
Cols : ['transaction_id', 'user_id', 'is_charge', 'price', 'created_at', 'message_status']
      transaction_id user_id is_charge price \
0  645842330555d60012bf9f1b  user_1      False      0
1  645a9c8ae9554b0019bc97ee  user_1      False      0
2  664b406166793d0019134356  user_1      False      0
```

```
      created_at message_status
0  0025-01-18 10:45:00 UTC      clicked
1  0025-01-07 12:02:00 UTC      clicked
2  0025-01-27 18:27:00 UTC      clicked
```

```
=====
FILE : Sms.csv
Shape : (20000, 8)
Cols : ['transaction_id', 'user_id', 'is_charge', 'price', 'total_sms', 'total_price', 'created_at', 'message_status']
      transaction_id user_id is_charge price total_sms total_price \
0  683e6cfe850e8b001240eb30  user_1      False      0      1      0
1  683fbb32850e8b001241a0d0  user_1      False      0      4      0
2  6840e5fe8cfe9700127dcd26  user_1      False      0      1      0
```

```
      created_at message_status
0  0025-01-08 17:09:00 UTC      failed
1  0025-01-27 10:15:00 UTC      failed
2  0025-02-20 13:51:00 UTC      failed
```

```
=====
FILE : Whatsapp.csv
Shape : (50000, 8)
```

```

Cols : ['transaction_id', 'user_id', 'message_type', 'price', 'last_status', 'is_charge', 'created_at', 'received_timestamp']
      transaction_id user_id message_type price last_status \
0 6819609cde80fc0012abe3e1 user_1 marketing 400 delivered
1 6838cb9a9eb51800120d7e4b user_1 marketing 400 delivered
2 683925b91e1cd70012699e41 user_1 marketing 400 delivered

      is_charge      created_at      received_timestamp
0      True  0025-02-14 09:27:00 UTC  0025-02-14 09:29:00 UTC
1      True  0025-01-02 16:21:00 UTC  0025-01-02 16:26:00 UTC
2      True  0025-03-18 09:20:00 UTC  0025-03-18 09:21:00 UTC

```

```

for name, df in dfs.items():
    print(f"\n{' '*40}")
    print(f"FILE : {name}")
    print(df.dtypes)
    print(f"Nulls:\n{df.isnull().sum()}")

```

```
=====
```

```

FILE : Call.csv
transaction_id    object
user_id           object
is_charge         bool
total_price       int64
created_at        object
message_status    object
dtype: object
Nulls:
transaction_id    0
user_id           0
is_charge         0
total_price       0
created_at        0
message_status    0
dtype: int64

```

```
=====
```

```

FILE : Email.csv
transaction_id    object
user_id           object
is_charge         bool
price             int64
created_at        object
message_status    object
dtype: object
Nulls:
transaction_id    0
user_id           0
is_charge         0
price             0
created_at        0
message_status    0
dtype: int64

```

```
=====
```

```

FILE : Sms.csv
transaction_id    object
user_id           object
is_charge         bool
price             int64
total_sms         int64
total_price       int64
created_at        object
message_status    object
dtype: object
Nulls:
transaction_id    0
user_id           0
is_charge         0
price             0
total_sms         0
total_price       0
created_at        0
message_status    0
dtype: int64

```

✦ Fix UTC TO WIB

```
# Kolom datetime per file
datetime_cols = {
    'Call.csv':      ['created_at'],
    'Email.csv':     ['created_at'],
    'Sms.csv':       ['created_at'],
    'Users.csv':     ['join_date'],
    'Whatsapp.csv':  ['created_at', 'received_timestamp'],
}
```

```
# Kolom datetime per file
for name, cols in datetime_cols.items():
    for col in cols:
        if name == 'Users.csv':
            dfs[name][col] = pd.to_datetime(dfs[name][col])
        else:
            # Fix typo tahun: 0025 → 2025
            dfs[name][col] = (
                dfs[name][col]
                .str.replace(r'^0025-', '2025-', regex=True)
                .pipe(pd.to_datetime, utc=True)
                .dt.tz_convert('Asia/Jakarta')
                .dt.tz_localize(None)
            )
```

```
# Cek
for name, df in dfs.items():
    print(f"\n{name}")
    print(df.select_dtypes(include='datetime').head(3))
```

```
Call.csv
      created_at
0 2025-01-29 16:13:00
1 2025-01-22 20:06:00
2 2025-02-26 00:48:00

Email.csv
      created_at
0 2025-01-18 17:45:00
1 2025-01-07 19:02:00
2 2025-01-28 01:27:00

Sms.csv
      created_at
0 2025-01-09 00:09:00
1 2025-01-27 17:15:00
2 2025-02-20 20:51:00

Whatsapp.csv
      created_at  received_timestamp
0 2025-02-14 16:27:00 2025-02-14 16:29:00
1 2025-01-02 23:21:00 2025-01-02 23:26:00
2 2025-03-18 16:20:00 2025-03-18 16:21:00

Users.csv
      join_date
0 2024-02-06
1 2024-12-19
2 2023-11-17
```

```
# Cek hasilnya
for name, df in dfs.items():
    print(f"\n{name}")
    print(df.dtypes)
```

```
Call.csv
transaction_id    object
user_id           object
is_charge         bool
total_price       int64
created_at        datetime64[ns]
message_status    object
dtype: object

Email.csv
transaction_id    object
user_id           object
is_charge         bool
```

```

price                int64
created_at           datetime64[ns]
message_status       object
dtype: object

Sms.csv
transaction_id       object
user_id              object
is_charge            bool
price                int64
total_sms            int64
total_price          int64
created_at           datetime64[ns]
message_status       object
dtype: object

Whatsapp.csv
transaction_id       object
user_id              object
message_type         object
price                int64
last_status          object
is_charge            bool
created_at           datetime64[ns]
received_timestamp   datetime64[ns]
dtype: object

Users.csv
user_id              object
user_name            object
field_of_business    object
join_date            datetime64[ns]
dtype: object

```

✎ Fix join_date to date

```

dfs['Users.csv']['join_date'] = pd.to_datetime(dfs['Users.csv']['join_date']).dt.date

print(dfs['Users.csv'].dtypes)
print(dfs['Users.csv'].head())

```

```

user_id              object
user_name            object
field_of_business    object
join_date            object
dtype: object

```

	user_id	user_name	field_of_business	join_date
0	user_8	PT Belajar Nusantara	Education	2024-02-06
1	user_5	PT Karya Sejahtera	Education	2024-12-19
2	user_17	PT Karya Jaya	Education	2023-11-17
3	user_15	PT Edu Mandiri	Education	2023-11-13
4	user_2	PT Dana Nusantara	Finance	2024-03-27

✎ EXPORT CSV - cleaned data

```

# import os

# OUTPUT_PATH = r"F:\5. Project\Jazmeen\RevouXTelkom\data\cleaned"
# os.makedirs(OUTPUT_PATH, exist_ok=True)

# for name, df in dfs.items():
#     output_file = os.path.join(OUTPUT_PATH, f"cleaned_{name}")
#     df.to_csv(output_file, index=False)
#     print(f"Saved: {output_file}")

```

```

# Gabung semua transaksi
dfs['Call.csv']['channel'] = 'Call'
dfs['Email.csv']['channel'] = 'Email'
dfs['Sms.csv']['channel'] = 'SMS'
dfs['Whatsapp.csv']['channel'] = 'WhatsApp'

# Kolom yang sama di semua file
cols = ['transaction_id', 'user_id', 'channel', 'created_at']

all_transactions = pd.concat([

```

```
dfs['Call.csv'][cols],
dfs['Email.csv'][cols],
dfs['Sms.csv'][cols],
dfs['Whatsapp.csv'][cols]
], ignore_index=True)

print(all_transactions.shape)
print(all_transactions['channel'].value_counts())
```

```
(122749, 4)
channel
WhatsApp    50000
Email        40000
SMS          20000
Call         12749
Name: count, dtype: int64
```

EDA untuk Clustering

```
# — Total volume & active days —————
vol = (all_transactions.groupby('user_id')
      .agg(total_messages=('transaction_id', 'count'),
           active_days=('created_at', lambda x: x.dt.date.nunique()))
      .reset_index())

vol['avg_msg_per_day'] = (vol['total_messages'] / vol['active_days']).round(2)

# — Total revenue —————
def get_revenue(df, price_col):
    df = df.copy()
    df['revenue'] = df.apply(
        lambda x: x[price_col] if x['is_charge'] else 0, axis=1)
    return df[['user_id', 'revenue']]

rev = pd.concat([
    get_revenue(dfs['Whatsapp.csv'], 'price'),
    get_revenue(dfs['Sms.csv'], 'total_price'),
    get_revenue(dfs['Email.csv'], 'price'),
    get_revenue(dfs['Call.csv'], 'total_price'),
])
rev = rev.groupby('user_id')['revenue'].sum().reset_index(name='total_revenue')

# — Channel distribution —————
ch_pivot = (all_transactions.groupby(['user_id', 'channel'])
            .size().unstack(fill_value=0).reset_index())
ch_pivot['total'] = ch_pivot[['WhatsApp', 'SMS', 'Email', 'Call']].sum(axis=1)
for ch in ['WhatsApp', 'SMS', 'Email', 'Call']:
    ch_pivot[f'{ch}_pct'] = (ch_pivot[ch] / ch_pivot['total'] * 100).round(1)

# — Gabung semua —————
profile = (vol
            .merge(rev, on='user_id')
            .merge(ch_pivot[['user_id', 'WhatsApp_pct', 'SMS_pct',
                             'Email_pct', 'Call_pct']], on='user_id')
            .merge(dfs['Users.csv'][['user_id', 'user_name', 'field_of_business']],
                  on='user_id'))

profile['rev_per_tx'] = (profile['total_revenue'] /
                        profile['total_messages']).round(2)

print(profile[['user_name', 'total_messages', 'total_revenue',
               'avg_msg_per_day', 'rev_per_tx']].sort_values(
    'total_messages', ascending=False).to_string(index=False))
```

	user_name	total_messages	total_revenue	avg_msg_per_day	rev_per_tx
	CV Wisata Indo	24635	5207360	270.71	211.38
	PT Dana Nusantara	24518	5198910	269.43	212.04
	CV Land Sejahtera	24496	5125530	269.19	209.24
	PT Konektivitas Indo	6230	1276070	68.46	204.83
	PT Karya Sejahtera	6198	1310570	68.11	211.45
	PT Retail Sejahtera	6132	1327690	67.38	216.52
	PT Belajar Nusantara	6117	1284620	67.22	210.01
	PT Logistik Mandiri	6010	1294970	66.04	215.47
	PT Estate Sejahtera	1593	339010	17.51	212.81
	CV Jalan Mandiri	1573	344080	17.29	218.74
	PT Travel Nusantara	1559	340730	17.32	218.56

CV Hotel Jaya	1554	342240	17.08	220.23
CV Niaga Indo	1552	336210	17.24	216.63
PT Saham Global	1530	335960	17.00	219.58
PT Mart Jaya	1525	323700	16.76	212.26
CV Hotel Nusantara	1523	305880	16.92	200.84
PT Citra Global	1519	317510	16.69	209.03
PT Karya Jaya	1510	315770	16.78	209.12
PT Edu Mandiri	1488	318840	16.53	214.27
CV Mart Indo	1487	300770	16.34	202.27

```

all_transactions['month'] = all_transactions['created_at'].dt.to_period('M')

monthly = (all_transactions.groupby(['user_id', 'month'])
            .size().unstack(fill_value=0).reset_index())

monthly.columns = [str(c) for c in monthly.columns]

monthly['decline_jan_feb'] = (
    (monthly['2025-01'] - monthly['2025-02']) / monthly['2025-01'] * 100
).round(1)

monthly['decline_feb_mar'] = (
    (monthly['2025-02'] - monthly['2025-03']) / monthly['2025-02'] * 100
).round(1)

# Klasifikasi tren (tanpa risk level, cukup arah)
def trend_label(row):
    if row['decline_jan_feb'] > 0 and row['decline_feb_mar'] > 10:
        return 'Declining'      # turun terus
    elif row['decline_jan_feb'] > 0 and row['decline_feb_mar'] <= 0:
        return 'Recovering'     # Jan turun, Mar naik
    else:
        return 'Stabilizing'    # melambat/flat

monthly['trend'] = monthly.apply(trend_label, axis=1)

churn = monthly[['user_id', '2025-01', '2025-02', '2025-03',
                'decline_jan_feb', 'decline_feb_mar', 'trend']]

result = (churn
            .merge(dfs['Users.csv'][['user_id', 'user_name', 'field_of_business']],
                  on='user_id')
            .sort_values('decline_feb_mar', ascending=False))

print(result.to_string(index=False))
print("\n=== Distribusi Tren ===")
print(monthly['trend'].value_counts())

```

user_id	2025-01	2025-02	2025-03	decline_jan_feb	decline_feb_mar	trend	user_name	field_of_business
user_11	626	528	405	15.7	23.3	Declining	PT Travel Nusantara	Hospitality
user_12	599	500	387	16.5	22.6	Declining	CV Mart Indo	Retail
user_19	604	514	434	14.9	15.6	Declining	CV Niaga Indo	eCommerce
user_14	598	490	435	18.1	11.2	Declining	CV Hotel Nusantara	Hospitality
user_20	615	496	441	19.3	11.1	Declining	CV Hotel Jaya	Hospitality
user_6	2407	1876	1719	22.1	8.4	Stabilizing	PT Logistik Mandiri	Logistics
user_9	585	484	449	17.3	7.2	Stabilizing	PT Citra Global	Hospitality
user_4	2521	1918	1784	23.9	7.0	Stabilizing	PT Konektivitas Indo	Telecommunications
user_13	633	464	433	26.7	6.7	Stabilizing	PT Saham Global	Finance
user_8	2441	1900	1772	22.2	6.7	Stabilizing	PT Belajar Nusantara	Education
user_2	9910	7527	7053	24.0	6.3	Stabilizing	PT Dana Nusantara	Finance
user_17	607	462	441	23.9	4.5	Stabilizing	PT Karya Jaya	Education
user_1	9703	7550	7225	22.2	4.3	Stabilizing	CV Land Sejahtera	Real Estate
user_3	9755	7594	7272	22.2	4.2	Stabilizing	CV Wisata Indo	Travel
user_7	2463	1861	1801	24.4	3.2	Stabilizing	PT Retail Sejahtera	Retail
user_5	2481	1875	1838	24.4	2.0	Stabilizing	PT Karya Sejahtera	Education
user_10	632	446	446	29.4	0.0	Recovering	PT Mart Jaya	Retail
user_15	594	437	457	26.4	-4.6	Recovering	PT Edu Mandiri	Education
user_16	658	452	482	31.3	-6.6	Recovering	PT Estate Sejahtera	Real Estate
user_18	630	454	488	27.9	-7.5	Recovering	CV Jalan Mandiri	Travel

```

=== Distribusi Tren ===
trend
Stabilizing    11
Declining       5
Recovering      4
Name: count, dtype: int64

```

Kalau dilihat penurunan volume transaksi dari bulan ke bulannya, hampir semua user/klien itu transaksinya mengalami penurunan dari total transaksinya (declining).

- Penurunan kurang dari 10% -> masih stabil
- Penurunan lebih dari 10% -> declining
- Penurunan minus (-), berarti ada kenaikan -> recovering

Dapat dilihat bahwa semua user/klien dari bulan Jan->Feb itu mengalami penurunan volume drastis, semua user menurun dari sekitar 15%-31% (bisa terjadi karena seasonal effect). Akan tetapi, dari bulan Feb-> Mar tidak semua user/klien mengalami penurunan volume transaksi/pengiriman pesan, ada beberapa user yang justru sempat mengalami kenaikan (recovering) walaupun tidak begitu signifikan/tinggi/ tidak sebanding dengan penurunannya.

Apabila penurunan terjadi karena alasan seasonal/musiman, itu diluar kendali dari user/klien nya juga, tidak bisa disalahkan dan tidak bisa di apa apain.

Kita tahu bahwa penyumbang revenue terbesar bagi OCA Blast ini ada di user_2, user_1, dan user_3 (backbone/penghasil utama income/revenue OCA) -> 59.9%, dilihat dari tren penurunan volume transaksi dari bulan ke bulannya dapat diketahui bahwa untuk user_1, user_2, dan user_3 ini masih stabil. Yang declining user_11, user_12, user_19, user_14, dan user_20 -> semuanya ada di segmen standard.

Balik lagi disini untuk tren/penamaan nya tidak ada threshold yang pasti / sesuai standar industri/lainnya, thresholdnya dibuat sendiri aturannya/menggunakan ruled-based sesuai data yang ada.

bagaimana caranya agar semua perhitungan ini atau analisis segmentasi ini bisa diterapkan di OCA / Platform serupa. Terkait volume tier kita masih mengelompokkannya dengan nilai eksak (di mana yang total messages nya itu lebih dari 20.000 masuk ke segmen 'Enterprise', lebih dari 5.000 masuk ke segmen 'Growth', dibawah 5.000 masuk ke segmen 'Standard'). Seiring berjalannya waktu total messagenya itu pasti bertambah terus (seharusnya). Daripada menggunakan nilai pasti untuk mengelompokkan/membuat segmen, lebih baik/disarankan dibuat per kuartil, Jadi, ada Q1, Q2, Q3, dan Q4, yang melewati Q3 itu masuk ke Enterprise, yang diatas Q2 masuk ke Growth, yang dibawah Q2 masuk Standard. Walaupun nanti bisnisnya berjalan terus menerus, tapi standarisasinya itu berdasarkan kuartil jadi tetap bisa diterapkan terus menerus segmentasinya.

```
all_transactions['month'] = all_transactions['created_at'].dt.to_period('M')

monthly = (all_transactions.groupby(['user_id', 'month'])
            .size().unstack(fill_value=0).reset_index())

monthly['baseline_avg'] = monthly[
    [c for c in monthly.columns if '2025-01' in str(c) or '2025-02' in str(c)]]
    .mean(axis=1)

monthly['vol_mar'] = monthly[
    [c for c in monthly.columns if '2025-03' in str(c)]]
    .sum(axis=1)

monthly['decline_pct'] = (
    (monthly['baseline_avg'] - monthly['vol_mar']) /
    monthly['baseline_avg'] * 100
).round(1)

def classify_risk(pct):
    if pct > 28: return 'High'
    if pct > 20: return 'Medium'
    if pct < -20: return 'Growing'
    return 'Stable'

monthly['risk_level'] = monthly['decline_pct'].apply(classify_risk)

churn = monthly[['user_id', 'baseline_avg', 'vol_mar', 'decline_pct', 'risk_level']]

print(churn.merge(dfs['Users.csv'][['user_id', 'user_name', 'field_of_business']],
                  on='user_id')
      .sort_values('decline_pct', ascending=False)
      .to_string(index=False))
```

user_id	baseline_avg	vol_mar	decline_pct	risk_level	user_name	field_of_business
user_11	577.0	405	29.8	High	PT Travel Nusantara	Hospitality
user_12	549.5	387	29.6	High	CV Mart Indo	Retail
user_19	559.0	434	22.4	Medium	CV Niaga Indo	eCommerce
user_13	548.5	433	21.1	Medium	PT Saham Global	Finance
user_20	555.5	441	20.6	Medium	CV Hotel Jaya	Hospitality
user_14	544.0	435	20.0	Stable	CV Hotel Nusantara	Hospitality
user_6	2141.5	1719	19.7	Stable	PT Logistik Mandiri	Logistics
user_4	2219.5	1784	19.6	Stable	PT Konektivitas Indo	Telecommunications
user_2	8718.5	7053	19.1	Stable	PT Dana Nusantara	Finance
user_8	2170.5	1772	18.4	Stable	PT Belajar Nusantara	Education
user_17	534.5	441	17.5	Stable	PT Karya Jaya	Education

user_10	539.0	446	17.3	Stable	PT Mart Jaya	Retail
user_7	2162.0	1801	16.7	Stable	PT Retail Sejahtera	Retail
user_1	8626.5	7225	16.2	Stable	CV Land Sejahtera	Real Estate
user_3	8674.5	7272	16.2	Stable	CV Wisata Indo	Travel
user_9	534.5	449	16.0	Stable	PT Citra Global	Hospitality
user_5	2178.0	1838	15.6	Stable	PT Karya Sejahtera	Education
user_16	555.0	482	13.2	Stable	PT Estate Sejahtera	Real Estate
user_15	515.5	457	11.3	Stable	PT Edu Mandiri	Education
user_18	542.0	488	10.0	Stable	CV Jalan Mandiri	Travel

Tambahan deep questions: adalah pertanyaan-pertanyaan yang bisa dijawab dari temuan-temuan pada data/hasil analisis

- Apakah penurunan volume terjadi / terkonsentrasi di industri tertentu? Hospitality
- volume decline / penurunan volume terjadi di channel mana saja, apakah merata di semua channel atau channel tertentu saja?
- Lihat pattern Call hour, di jam brp sering rna dan answered, rekomendasi jam answered, more answered more revenue

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X = profile[['total_messages', 'total_revenue']].copy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Cari k optimal
print("K | Inertia | Silhouette")
for k in range(2, 7):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    sil = silhouette_score(X_scaled, km.labels_)
    print(f"{k} | {km.inertia_:7.2f} | {sil:.3f}")

# Final k=3
km3 = KMeans(n_clusters=3, random_state=42, n_init=10)
profile['cluster'] = km3.fit_predict(X_scaled)

# Label cluster berdasarkan avg volume (terbesar = Enterprise)
order = (profile.groupby('cluster')['total_messages']
        .mean().sort_values(ascending=False))
label_map = {order.index[0]: 'Enterprise',
             order.index[1]: 'Growth',
             order.index[2]: 'Standard'}
profile['segment'] = profile['cluster'].map(label_map)

print(profile[['user_name', 'segment', 'total_messages', 'total_revenue']]
      .sort_values('total_messages', ascending=False).to_string(index=False))
```

```
K | Inertia | Silhouette
2 | 2.36 | 0.912
3 | 0.00 | 0.983
4 | 0.00 | 0.902
5 | 0.00 | 0.701
6 | 0.00 | 0.534
```

	user_name	segment	total_messages	total_revenue
	CV Wisata Indo	Enterprise	24635	5207360
	PT Dana Nusantara	Enterprise	24518	5198910
	CV Land Sejahtera	Enterprise	24496	5125530
	PT Konektivitas Indo	Growth	6230	1276070
	PT Karya Sejahtera	Growth	6198	1310570
	PT Retail Sejahtera	Growth	6132	1327690
	PT Belajar Nusantara	Growth	6117	1284620
	PT Logistik Mandiri	Growth	6010	1294970
	PT Estate Sejahtera	Standard	1593	339010
	CV Jalan Mandiri	Standard	1573	344080
	PT Travel Nusantara	Standard	1559	340730
	CV Hotel Jaya	Standard	1554	342240
	CV Niaga Indo	Standard	1552	336210
	PT Saham Global	Standard	1530	335960
	PT Mart Jaya	Standard	1525	323700
	CV Hotel Nusantara	Standard	1523	305880
	PT Citra Global	Standard	1519	317510
	PT Karya Jaya	Standard	1510	315770
	PT Edu Mandiri	Standard	1488	318840
	CV Mart Indo	Standard	1487	300770

```
# Gabung churn ke profile
seg = profile.merge(churn[['user_id', 'decline_pct', 'risk_level']], on='user_id')
```



```
# Ringkasan per segmen
summary = seg.groupby('segment').agg(
    n_clients      = ('user_id',      'count'),
    avg_volume     = ('total_messages', 'mean'),
    avg_revenue    = ('total_revenue', 'mean'),
    total_revenue  = ('total_revenue', 'sum'),
    avg_msg_day    = ('avg_msg_per_day', 'mean'),
    avg_rev_tx     = ('rev_per_tx',    'mean'),
    avg_decline    = ('decline_pct',   'mean'),
    high_risk      = ('risk_level',    lambda x: (x == 'High').sum()),
    medium_risk    = ('risk_level',    lambda x: (x == 'Medium').sum()),
).round(1)

summary['rev_share_%'] = (
    summary['total_revenue'] / summary['total_revenue'].sum() * 100
).round(1)

print(summary.reindex(['Enterprise', 'Growth', 'Standard']).to_string())
```

	n_clients	avg_volume	avg_revenue	total_revenue	avg_msg_day	avg_rev_tx	avg_decline	high_risk	medium_risk	rev
segment										
Enterprise	3	24549.7	5177266.7	15531800	269.8	210.9	17.2	0	0	
Growth	5	6137.4	1298784.0	6493920	67.4	211.7	18.0	0	0	
Standard	12	1534.4	326725.0	3920700	17.0	212.9	19.1	2	3	

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import numpy as np

colors = {'Enterprise': '#E74C3C', 'Growth': '#F39C12', 'Standard': '#27AE60'}

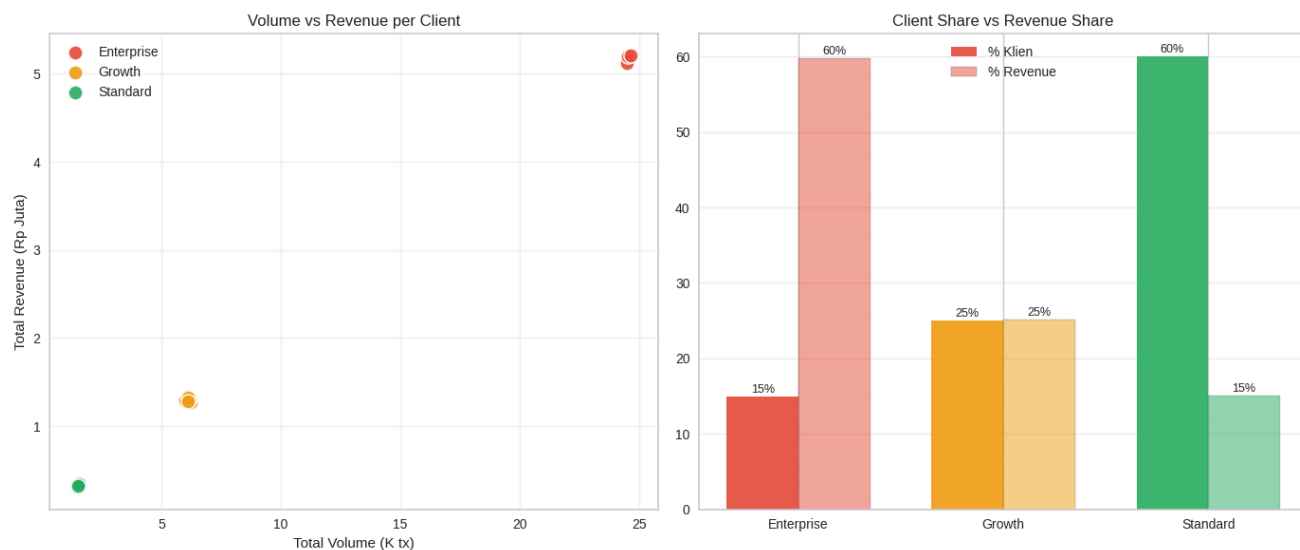
# — Viz 1: Scatter Volume vs Revenue —————
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

ax = axes[0]
for s, grp in seg.groupby('segment'):
    ax.scatter(grp['total_messages']/1000, grp['total_revenue']/1e6,
               c=colors[s], label=s, s=120,
               edgecolors='white', linewidth=1.2, alpha=0.9)
ax.set_xlabel('Total Volume (K tx)')
ax.set_ylabel('Total Revenue (Rp Juta)')
ax.set_title('Volume vs Revenue per Client')
ax.legend(); ax.grid(alpha=0.25)

# — Viz 2: Client share vs Revenue share —————
ax = axes[1]
segs = ['Enterprise', 'Growth', 'Standard']
client_pct = [15, 25, 60]
revenue_pct = [59.8, 25.1, 15.1]
x = np.arange(3); w = 0.35

b1 = ax.bar(x-w/2, client_pct, w, color=[colors[s] for s in segs],
            alpha=0.9, label='% Klien')
b2 = ax.bar(x+w/2, revenue_pct, w, color=[colors[s] for s in segs],
            alpha=0.5, edgecolor='black', ls='--', label='% Revenue')
ax.set_xticks(x); ax.set_xticklabels(segs)
ax.set_title('Client Share vs Revenue Share')
ax.legend(); ax.grid(axis='y', alpha=0.3)
for b in list(b1)+list(b2):
    ax.text(b.get_x()+b.get_width()/2, b.get_height()+0.5,
            f'{b.get_height():.0f}%', ha='center', fontsize=9)

plt.tight_layout()
plt.savefig('scatter_revenue.png', dpi=150, bbox_inches='tight')
plt.show()
```



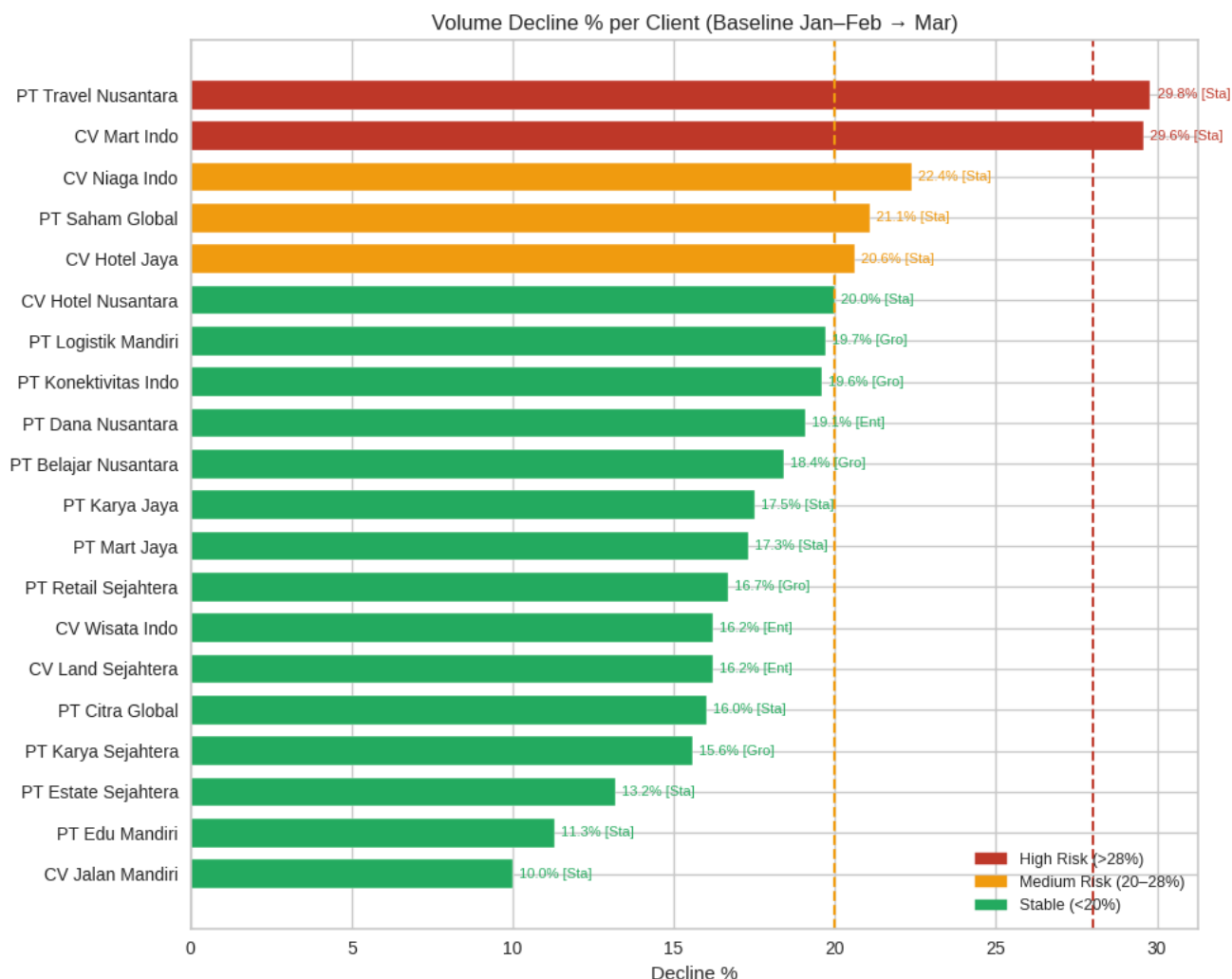
```
# — Viz 3: Churn bar per client —————
RISK_COLORS = {'High': '#C0392B', 'Medium': '#F39C12',
               'Stable': '#27AE60', 'Growing': '#2980B9'}

fig, ax = plt.subplots(figsize=(10, 8))
sorted_df = seg.sort_values('decline_pct', ascending=True)
risk_clr = [RISK_COLORS.get(r, '#999') for r in sorted_df['risk_level']]

bars = ax.barh(sorted_df['user_name'], sorted_df['decline_pct'],
               color=risk_clr, edgecolor='white', height=0.7)
ax.axvline(28, color='#C0392B', ls='--', lw=1.5, label='High threshold (28%)')
ax.axvline(20, color='#F39C12', ls='--', lw=1.5, label='Medium threshold (20%)')

for bar, (_, row) in zip(bars, sorted_df.iterrows()):
    ax.text(bar.get_width()+0.2, bar.get_y()+bar.get_height()/2,
           f'{row["decline_pct"]:.1f}% [{row["segment"][:3]}]',
           va='center', fontsize=8,
           color=RISK_COLORS.get(row['risk_level'], '#999'))

legend_handles = [
    mpatches.Patch(color='#C0392B', label='High Risk (>28%)'),
    mpatches.Patch(color='#F39C12', label='Medium Risk (20-28%)'),
    mpatches.Patch(color='#27AE60', label='Stable (<20%)'),
]
ax.legend(handles=legend_handles, fontsize=9)
ax.set_title('Volume Decline % per Client (Baseline Jan-Feb → Mar)')
ax.set_xlabel('Decline %')
plt.tight_layout()
plt.savefig('churn_risk.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
# — Viz 4: Hourly Heatmap per Segment —————
all_transactions['hour'] = all_transactions['created_at'].dt.hour
all_transactions['weekday'] = all_transactions['created_at'].dt.day_name()

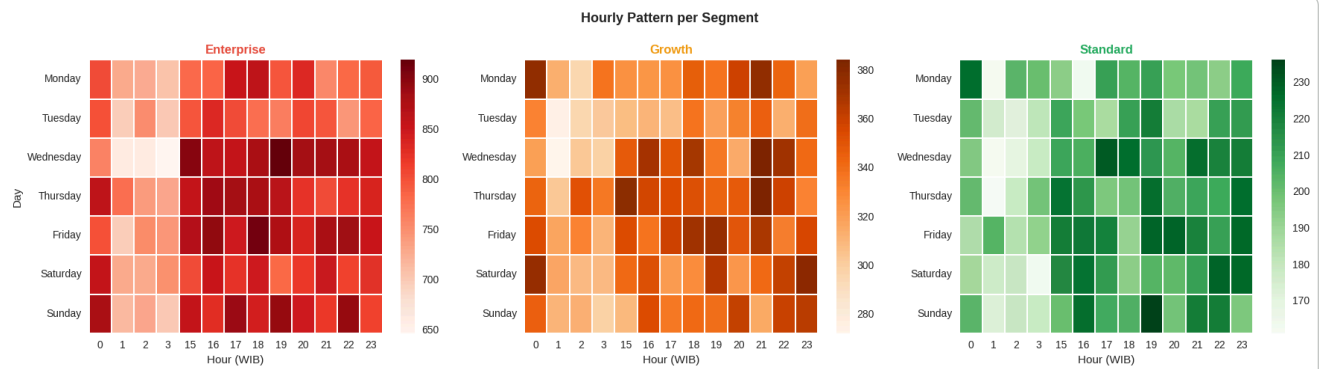
seg_map = seg.set_index('user_id')['segment']
all_transactions['segment'] = all_transactions['user_id'].map(seg_map)

DAY_ORDER = ['Monday', 'Tuesday', 'Wednesday',
              'Thursday', 'Friday', 'Saturday', 'Sunday']
CMAPS = {'Enterprise': 'Reds', 'Growth': 'Oranges', 'Standard': 'Greens'}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Hourly Pattern per Segment', fontsize=13, fontweight='bold')

for i, s in enumerate(['Enterprise', 'Growth', 'Standard']):
    ax = axes[i]
    sub = all_transactions[all_transactions['segment'] == s].copy()
    pivot = (sub.groupby(['weekday', 'hour']).size()
              .unstack(fill_value=0)
              .reindex([d for d in DAY_ORDER
                        if d in sub['weekday'].unique()]))
    sns.heatmap(pivot, ax=ax, cmap=CMAPS[s],
                linewidths=0.3, linecolor='white')
    ax.set_title(s, fontweight='bold', color=colors[s])
    ax.set_xlabel('Hour (WIB)')
    ax.set_ylabel('Day' if i == 0 else '')

plt.tight_layout()
plt.savefig('hourly_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
print("=" * 55)
print("DEEP QUESTIONS – DIJAWAB DARI DATA")
print("=" * 55)

print("\nQ1: Which segment is most profitable?")
q1 = (seg.groupby('segment')['total_revenue']
      .sum().reset_index())
q1['pct'] = (q1['total_revenue'] / q1['total_revenue'].sum() * 100).round(1)
print(q1.sort_values('total_revenue', ascending=False).to_string(index=False))

print("\nQ2: Which clients are most prone to churn?")
print(seg[seg['risk_level'].isin(['High', 'Medium'])
        [['user_name', 'field_of_business', 'segment',
          'decline_pct', 'risk_level']]
      .sort_values('decline_pct', ascending=False)
      .to_string(index=False))

print("\nQ3: Unique hourly patterns per tier?")
print(" → Lihat heatmap")
print(" Enterprise : 16-18 WIB (B2B corporate)")
print(" Growth      : 21:00 & 00:00 WIB")
print(" Standard     : 19-23 WIB (evening consumer)")

print("\nQ4: Mono-channel upsell potential?")
print(" Semua klien sudah 4/4 channel → tidak applicable")
print(" Reframed: upsell dari VOLUME")
print(" Standard → Growth : 17/hari → 67/hari")
print(" Growth → Enterprise: 67/hari → 270/hari")
```

```
=====
DEEP QUESTIONS – DIJAWAB DARI DATA
=====
```

Q1: Which segment is most profitable?

segment	total_revenue	pct
Enterprise	15531800	59.9
Growth	6493920	25.0
Standard	3920700	15.1

Q2: Which clients are most prone to churn?

user_name	field_of_business	segment	decline_pct	risk_level
PT Travel Nusantara	Hospitality	Standard	29.8	High
CV Mart Indo	Retail	Standard	29.6	High
CV Niaga Indo	eCommerce	Standard	22.4	Medium
PT Saham Global	Finance	Standard	21.1	Medium
CV Hotel Jaya	Hospitality	Standard	20.6	Medium

Q3: Unique hourly patterns per tier?

→ Lihat heatmap

Enterprise : 16-18 WIB (B2B corporate)

Growth : 21:00 & 00:00 WIB

Standard : 19-23 WIB (evening consumer)

Q4: Mono-channel upsell potential?

Semua klien sudah 4/4 channel → tidak applicable

Reframed: upsell dari VOLUME

Standard → Growth : 17/hari → 67/hari

Growth → Enterprise: 67/hari → 270/hari

Churn Risk Analysis

Pendekatan

Tiga pendekatan digunakan untuk mendeteksi potensi churn, diterapkan secara berurutan:

1. Days Inactive (Last Activity Gap)

Mengukur berapa hari sejak transaksi terakhir user hingga tanggal akhir data.

Formula: $\text{days_inactive} = \text{last_date} - \text{last_transaction_date}$

$\text{churn_risk} = \text{True}$ jika $\text{days_inactive} > 14$

Hasil: Semua user menunjukkan $\text{days_inactive} = 0$, karena seluruh 20 user masih aktif hingga hari terakhir data (1 April 2025). Pendekatan ini tidak menghasilkan sinyal churn yang berguna untuk dataset ini.

2. Zero Weeks (Calendar Gap Detection)

Mendeteksi apakah user pernah tidak mengirim pesan sama sekali dalam satu minggu kalender penuh di tengah periode.

Formula:

$\text{zero_weeks} = \text{jumlah minggu di mana msg_count} = 0$

$\text{dropout_risk} = \text{True}$ jika $\text{zero_weeks} > 2$

Hasil: Semua user memiliki $\text{zero_weeks} = 0$, artinya tidak ada gap mingguan yang terdeteksi. Seluruh user aktif setiap minggu sepanjang periode Jan–Mar 2025.

3. Volume Decline (Baseline vs Recent)

Membandingkan rata-rata volume pengiriman pesan per minggu antara periode baseline dan periode terkini. Ini adalah pendekatan yang paling sensitif dan relevan untuk dataset ini.

Preprocessing:

- Minggu terakhir (Mar 31 – Apr 6) di-exclude karena tidak lengkap (hanya 2 hari, 720 transaksi vs normal ~8.200/minggu), sehingga total minggu yang digunakan adalah 13 minggu.

Window:

Total minggu lengkap : 13 minggu (Jan 2025 – Mar 30 2025)

Baseline : 9 minggu pertama

Recent : 4 minggu terakhir

Formula:

$\text{baseline_avg} = \text{rata-rata msg/minggu selama 9 minggu baseline}$

$\text{recent_avg} = \text{rata-rata msg/minggu selama 4 minggu recent}$

$\text{decline_pct} = (\text{baseline_avg} - \text{recent_avg}) / \text{baseline_avg} \times 100\%$

```
from datetime import timedelta

# Tanggal terakhir di data
last_date = all_transactions['created_at'].max()

# Transaksi terakhir per user
last_activity = all_transactions.groupby('user_id')['created_at'].max().reset_index()
last_activity.columns = ['user_id', 'last_transaction']

# Hitung gap hari
last_activity['days_inactive'] = (last_date - last_activity['last_transaction']).dt.days

# Flag churn (inactive > 14 hari)
last_activity['churn_risk'] = last_activity['days_inactive'] > 14

print(last_activity.sort_values('days_inactive', ascending=False))
```

```
   user_id  last_transaction  days_inactive  churn_risk
0   user_1  2025-04-01 03:59:00           0         False
```

```

1 user_10 2025-04-01 00:20:00 0 False
2 user_11 2025-03-31 23:24:00 0 False
3 user_12 2025-04-01 01:06:00 0 False
4 user_13 2025-03-31 22:43:00 0 False
5 user_14 2025-03-31 20:13:00 0 False
6 user_15 2025-03-31 21:45:00 0 False
7 user_16 2025-04-01 01:11:00 0 False
8 user_17 2025-03-31 23:16:00 0 False
9 user_18 2025-04-01 02:51:00 0 False
10 user_19 2025-03-31 22:41:00 0 False
11 user_2 2025-04-01 03:55:00 0 False
12 user_20 2025-04-01 00:28:00 0 False
13 user_3 2025-04-01 03:50:00 0 False
14 user_4 2025-04-01 02:32:00 0 False
15 user_5 2025-04-01 03:48:00 0 False
16 user_6 2025-04-01 02:56:00 0 False
17 user_7 2025-04-01 03:43:00 0 False
18 user_8 2025-04-01 03:35:00 0 False
19 user_9 2025-04-01 02:18:00 0 False

```

```

# Hitung volume per user per minggu
all_transactions['week'] = all_transactions['created_at'].dt.to_period('W')

weekly_volume = all_transactions.groupby(['user_id', 'week']).size().reset_index(name='msg_count')

# Cek user yang ada minggu kosong (tidak kirim sama sekali)
all_weeks = weekly_volume['week'].unique()

user_weeks = weekly_volume.pivot_table(index='user_id', columns='week', values='msg_count', fill_value=0)

# Flag user yang punya minggu dengan 0 pesan
user_weeks['zero_weeks'] = (user_weeks == 0).sum(axis=1)
user_weeks['dropout_risk'] = user_weeks['zero_weeks'] > 2

print(user_weeks[['zero_weeks', 'dropout_risk']].sort_values('zero_weeks', ascending=False))

```

week	zero_weeks	dropout_risk
user_id		
user_1	0	False
user_10	0	False
user_11	0	False
user_12	0	False
user_13	0	False
user_14	0	False
user_15	0	False
user_16	0	False
user_17	0	False
user_18	0	False
user_19	0	False
user_2	0	False
user_20	0	False
user_3	0	False
user_4	0	False
user_5	0	False
user_6	0	False
user_7	0	False
user_8	0	False
user_9	0	False

```

print(f"Last date in data: {last_date}")
print(f"\nDate range: {all_transactions['created_at'].min()} to {all_transactions['created_at'].max()}")

```

Last date in data: 2025-04-01 03:59:00

Date range: 2025-01-01 15:00:00 to 2025-04-01 03:59:00

✓ Churn Detection — Volume Decline

Karena semua user masih aktif sampai tanggal terakhir, churn berbasis recency tidak relevan. Pendekatan: bandingkan rata-rata volume mingguan di **4 minggu terakhir** vs **periode baseline sebelumnya**.

- `decline_pct > 40%` → **High risk** (churn)
- `decline_pct 20-40%` → **Medium risk** (warning)
- `decline_pct < 20%` → **Stable**

```

# -----
# CHURN: Volume Decline (last 4 weeks vs baseline)
# -----
RECENT_WEEKS      = 4
HIGH_RISK_THRESHOLD = 40
MED_RISK_THRESHOLD = 20

# Volume mingguan per user
weekly = (
    all_transactions
    .groupby(['user_id', 'week'])
    .size()
    .reset_index(name='msg_count')
)

pivot = weekly.pivot_table(
    index='user_id', columns='week', values='msg_count', fill_value=0
)

# Exclude minggu terakhir (tidak lengkap)
complete_weeks = sorted(pivot.columns)[-1]
recent_weeks    = complete_weeks[-RECENT_WEEKS:]
baseline_weeks  = complete_weeks[: -RECENT_WEEKS]

print(f"Baseline: {baseline_weeks[0]} → {baseline_weeks[-1]} ({len(baseline_weeks)} minggu)")
print(f"Recent  : {recent_weeks[0]} → {recent_weeks[-1]} ({len(recent_weeks)} minggu)")

# Kalkulasi decline
churn2 = pd.DataFrame({
    'baseline_avg': pivot[baseline_weeks].mean(axis=1),
    'recent_avg'  : pivot[recent_weeks].mean(axis=1),
})
churn2['decline_pct'] = np.where(
    churn2['baseline_avg'] > 0,
    (churn2['baseline_avg'] - churn2['recent_avg']) / churn2['baseline_avg'] * 100,
    0
)

def classify(pct):
    if pct > HIGH_RISK_THRESHOLD: return 'High'
    if pct > MED_RISK_THRESHOLD:  return 'Medium'
    if pct < -MED_RISK_THRESHOLD: return 'Growing'
    return 'Stable'

churn2['risk_level'] = churn2['decline_pct'].apply(classify)

# Gabung info user
users_df = dfs['Users.csv'][['user_id', 'user_name', 'field_of_business']]
churn2 = churn2.reset_index().merge(users_df, on='user_id', how='left')
churn2 = churn2.sort_values('decline_pct', ascending=False)

print("\nDistribusi risk level:")
print(churn2['risk_level'].value_counts())
print(churn2[['user_id', 'user_name', 'field_of_business', 'baseline_avg', 'recent_avg', 'decline_pct', 'risk_level']])

```

Baseline: 2024-12-30/2025-01-05 → 2025-02-24/2025-03-02 (9 minggu)
Recent : 2025-03-03/2025-03-09 → 2025-03-24/2025-03-30 (4 minggu)

Distribusi risk level:

risk_level

Stable 15

Medium 5

Name: count, dtype: int64

	user_id	user_name	field_of_business	baseline_avg	\
2	user_11	PT Travel Nusantara	Hospitality	131.111111	
3	user_12	CV Mart Indo	Retail	124.666667	
5	user_14	CV Hotel Nusantara	Hospitality	125.333333	
4	user_13	PT Saham Global	Finance	125.111111	
10	user_19	CV Niaga Indo	eCommerce	126.888889	
16	user_6	PT Logistik Mandiri	Logistics	488.888889	
14	user_4	PT Konektivitas Indo	Telecommunications	505.444444	
11	user_2	PT Dana Nusantara	Finance	1988.444444	
8	user_17	PT Karya Jaya	Education	122.000000	
12	user_20	CV Hotel Jaya	Hospitality	125.555556	
18	user_8	PT Belajar Nusantara	Education	494.555556	
19	user_9	PT Citra Global	Hospitality	122.000000	
0	user_1	CV Land Sejahtera	Real Estate	1971.888889	
13	user_3	CV Wisata Indo	Travel	1981.777778	
1	user_10	PT Mart Jaya	Retail	122.555556	

```

15 user_5 PT Karya Sejahtera Education 496.333333
17 user_7 PT Retail Sejahtera Retail 491.444444
7 user_16 PT Estate Sejahtera Real Estate 127.000000
6 user_15 PT Edu Mandiri Education 117.555556
9 user_18 CV Jalan Mandiri Travel 124.222222

recent_avg decline_pct risk_level
2 92.75 29.258475 Medium
3 89.00 28.609626 Medium
5 97.00 22.606383 Medium
4 97.50 22.069272 Medium
10 100.75 20.599825 Medium
16 394.75 19.255682 Stable
14 410.25 18.833810 Stable
11 1620.50 18.504135 Stable
8 100.00 18.032787 Stable
12 103.50 17.566372 Stable
18 409.25 17.248933 Stable
19 102.00 16.393443 Stable
0 1652.50 16.197104 Stable
13 1665.25 15.971911 Stable
1 103.75 15.344515 Stable
15 421.50 15.077233 Stable
17 417.75 14.995478 Stable
7 110.50 12.992126 Stable
6 103.75 11.743856 Stable
9 111.25 10.442755 Stable

```

▼ Project 2 : Active User Behavior & Segmentation

▼ Import Library


```
# Project 2 : Active User Behavior & Segmentation
# Import Libraries
import pandas as pd
# menambahkan library pandas (untuk manipulasi data dalam bentuk DataFrame)

import numpy as np
# menambahkan library numpy (untuk operasi numerik dan array)

from datetime import datetime
# mengambil fungsi datetime (untuk bekerja dengan tanggal & waktu)

import seaborn as sns
# menambahkan library seaborn (untuk visualisasi data statistik)

import matplotlib.pyplot as plt
# menambahkan modul pyplot (untuk membuat grafik)

import matplotlib.image as mpimg
# menambahkan modul image (untuk membaca/menampilkan gambar)

import matplotlib.cm as cm
# menambahkan colormap module (untuk skema warna pada visualisasi)

from yellowbrick.cluster import SilhouetteVisualizer
# menambahkan alat visualisasi siluet untuk analisis k-means clustering

from sklearn.cluster import KMeans # K-Means Clustering
# menambahkan algoritma K-Means (untuk clustering/segmentasi data)

from sklearn.preprocessing import MinMaxScaler, MaxAbsScaler, StandardScaler, RobustScaler, PowerTransformer, QuantileTransformer
# menambahkan berbagai scaler dan transformer untuk normalisasi/transformasi data

from sklearn import metrics
# menambahkan modul evaluasi model (akurasi, F1, silhouette, dll)

from sklearn import cluster
# menambahkan modul clustering lain (DBSCAN, Agglomerative, MeanShift)

from sklearn.metrics import silhouette_samples, silhouette_score
# menambahkan fungsi perhitungan silhouette untuk mengevaluasi kualitas clustering

from sklearn.datasets import make_blobs
# menambahkan fungsi untuk membuat data dummy/ synthetic cluster (Gaussian blobs)
```

```
# Pengaturan untuk menghilangkan warning (biar notebook bersih saja, optional)
import warnings
warnings.filterwarnings('ignore')
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')
print("✅ Libraries loaded")
```

✅ Libraries loaded

▼ Project 2 Brief : Active User Behavior & Segmentation

Project 1 kita telah membuat unified dashboard yang dapat memonitoring performa bisnis all accross channels yang terdapat pada OCA (WA, Email, SMS, dan Call). Temuan kritis yang didapatkan diantaranya:

- Top 3 klien dari 3 industri berbeda (user_3 (Travel), user_2 (Finance), dan user_3(Real Estate)) mendominasi volume pesan/transaksi dan revenue baik dari sisi total revenue dan juga total volume message.
- Berdasarkan Churn Risk Analysis (Churn Detection) menggunakan persentase volume decline untuk masing-masing user, diketahui bahwa terdapat 5 klien OCA menunjukkan medium churn risk (baseline volume decline mencapai 20-29%)

- Dashboard menunjukkan frekuensi (active days) hampir identik untuk semua klien (90–91 hari). Artinya, klien OCA tidak ada yang benar-benar inaktif > 14 hari/dormant. Diferensiasi klien tidak bisa dilihat dari frekuensi nya (berapa hari user-user tersebut aktif). Karena semua klien oca aktif tiap harinya selama periode Jan–Mar 2025
- Mono-channel upsell tidak terlalu applicable (tidak bisa diterapkan) karena semua user/client oca sudah menggunakan 4/4 channel yang ada dan semua user dominan menggunakan WhatsApp, sehingga upselling harus dilihat dari dimensi lain bukan dari penggunaan channel.

Lanjut ke project 2, pada project 2 ini akan dilakukan segmentasi user/client berdasarkan perilaku transaksi yang dapat dibedakan untuk 20 klien OCA

OCA Blast adalah platform B2B CPaaS (Communication Platform as-a-Service) milik Telkom Indonesia yang menyediakan layanan pengiriman pesan secara massal atau dalam jumlah yang banyak, klien OCA Blast adalah perusahaan/institusi/korporasi, bukan individu/perorangan. OCA Blast menyediakan empat channel untuk pengiriman pesan, yaitu Whatsapp, Email, SMS, dan Call.

- Terdapat 20 klien oca yang aktif
- Total transaksi mencakup semua channel mencapai angka 122.749 transaksi/volume pesan
- Total Revenue OCA mencapai Rp25,95 Juta
- Persebaran volume message (pesan) per user (max: 24.635 transaksi, min: 1.487 transaksi) atau sekitar 16.6 kali lipat
- Persebaran total revenue per user (max: Rp5,2 Juta, min: Rp300 ribu transaksi) atau sekitar 17 kali lipat
- Semua klien aktif setiap hari, 4/4 channel

total message dan total revenue adalah differentiator yang paling dapat digunakan untuk segmentasi

Implikasinya:

- Satu klien churn = kehilangan ribuan transaksi sekaligus
- Perilaku penggunaan tiap klien mencerminkan kebutuhan bisnis yang berbeda
- Strategi retention dan upsell harus disesuaikan per profil klien, bukan secara general

Problem nya:

- Tidak ada segmentasi yang mencerminkan/menggambarkan/ yang dapat memahami karakteristik / perilaku aktual untuk 20 klien OCA Blast yang menggunakan 4 channel (WA, SMS, Email, Call). Seluruh klien diperlakukan sama meski volume penggunaan berbeda (1.487 tx vs 24.635 tx) dan revenue berbeda (Rp 300K vs Rp 5.2M/periode). Masalah spesifik: (1) tidak diketahui klien mana yang paling valuable, (2) tidak ada early warning untuk klien yang volume-nya menurun, (3) tidak ada strategi upsell yang disesuaikan per kelompok klien.

Baseline volume decline_pct = Analisis berbasis data Jan–Mar 2025 (90–91 hari, periode penuh). Segmentasi menggunakan 3 bulan penuh sebagai baseline (April diexclude karena hanya 3 hari data). Monthly trend dihitung dari baseline (Jan+Feb)/2 dibandingkan Mar 2025. Hasil segmentasi ditargetkan dapat diimplementasikan sebagai dasar strategi Q2 2025 (Apr–Jun).

Tujuan Segmentasi User/Client OCA Blast:

1. Personalization & Prioritization : Untuk menjawab "siapa klien terpenting dan perlu diprioritaskan?" Tujuannya adalah untuk memberikan perlakuan (treatment) yang dipersonalisasi, artinya memberikan perlakuan yang tepat kepada klien yang tepat dan sesuai dengan kebutuhan masing-masing segmen.
 2. Churn Detection & Revenue Protection : Untuk menjawab "siapa klien yang berisiko churn?" Mengidentifikasi klien berisiko sebelum mereka benar-benar churn. Revenue OCA Blast sangat terkonsentrasi kepada top 3 user berdasarkan volume dan revenue yang dihasilkannya.
 3. Upsell Strategy by Segment : Untuk menjawab "siapa klien yang bisa di upsell/ditingkatkan?" Menentukan strategi upsell yang berbeda per segmen. Bukan dari channel diversity (semua user sudah 4/4 channel), tapi bisa dari segi rekomendasi channel mix / peningkatan volume dan frekuensi blast yang disesuaikan dengan karakteristik tiap tier.
 4. Deep Questions Analysis (Insight): Menjawab 4 pertanyaan analitik kritis dari brief: profitabilitas per segmen, identifikasi klien paling rentan churn, pola hourly unik per segmen, dan potensi upsell berdasarkan channel usage.
- Tujuan 1 — Behavioral Profiling: Memahami profil perilaku / pola penggunaan platform untuk setiap 20 klien aktif berdasarkan volume pesan/volume transaksi, total revenue yang dihasilkan per user, intensitas harian penggunaan layanan (avg message per day), efisiensi monetisasi per transaksi (rev_per_tx), dan hourly pattern selama Jan–Mar 2025.
 - Tujuan 2 — Revenue Protection: Mengidentifikasi klien yang berkontribusi paling besar terhadap Rp25,95 juta revenue kuartalan untuk diprioritaskan dalam program retensi.
 - Tujuan 3 — Churn Early Warning: Mendeteksi klien yang menunjukkan tren penurunan volume (Jan→Mar) sebelum benar-benar churn, berdasarkan sinyal behavioral dari data historis.

- Tujuan 4 — Differentiated Strategy: Menyusun dan memberikan rekomendasi targeting dan treatment yang berbeda per segmen, bukan satu strategi yang sama untuk semua klien agar intervensi Account Manager lebih tepat sasaran dan efisien. Bukan insight generik tapi strategi spesifik berdasarkan karakteristik klien.
- Tujuan 5 — Upsell Identification: Mengidentifikasi klien yang underutilize channel SMS dan Call (revenue per transaksi lebih tinggi) untuk ditawarkan volume upsell.

Problem Statement:

"Bagaimana OCA Blast dapat mengidentifikasi dan mengelompokkan 20 klien aktif B2B-nya berdasarkan pola perilaku penggunaan platform (volume transaksi, intensitas harian, dan efisiensi revenue) selama Jan–Mar 2025, sehingga dapat merancang strategi retensi dan upsell yang terdiferensiasi per segmen untuk mempertahankan Rp25,95 juta revenue kuartalan dan mencegah eskalasi churn risk sebelum Q2 2025 berakhir?"

Problem Statement

OCA Blast memiliki 20 klien korporat aktif yang menggunakan platform mass messaging B2B selama periode Jan–Mar 2025, menghasilkan 122.749 transaksi dan Rp 25.95 Juta revenue. Namun, tidak ada segmentasi klien berbasis perilaku aktual untuk 20 klien aktif OCA Blast yang dapat digunakan untuk menentukan strategi retensi, upsell, dan alokasi sumber daya yang berbeda per kelompok klien. Akibatnya, seluruh 20 klien diperlakukan dengan pendekatan yang sama, padahal terdapat gap yang besar dalam volume pesan / volume transaksi dan dalam kontribusi revenue antar klien, kondisi ini meningkatkan risiko churn yang tidak terdeteksi dini dan peluang upsell yang tidak dioptimalkan.

✓ User Profiling

Berikut terdapat pula gambaran dan hasil jika menggunakan RFM / Rule-Based untuk Segmentasi User/Client OCA Blast

```
import pandas as pd
import numpy as np

# -----
# PROJECT 2 – Active User Behavior & Segmentation
# -----

# Pastikan all_transactions sudah ada dari Project 1
# dengan kolom: transaction_id, user_id, channel, created_at

# -----
# 1. USER PROFILE – Volume, Frequency, Recency
# -----

last_date = all_transactions['created_at'].max()

user_profile = all_transactions.groupby('user_id').agg(
    total_messages = ('transaction_id', 'count'),
    active_days     = ('created_at', lambda x: x.dt.date.nunique()),
    first_transaction= ('created_at', 'min'),
    last_transaction = ('created_at', 'max'),
    channel_count    = ('channel', 'nunique'),
    channels_used     = ('channel', lambda x: ', '.join(sorted(x.unique())))
).reset_index()

user_profile['days_inactive'] = (last_date - user_profile['last_transaction']).dt.days
user_profile['tenure_days']    = (user_profile['last_transaction'] - user_profile['first_transaction']).dt.days
user_profile['avg_msg_per_day'] = (user_profile['total_messages'] / user_profile['active_days']).round(1)

print("=== User Profile Overview ===")
print(user_profile[['user_id', 'total_messages', 'active_days', 'channel_count', 'avg_msg_per_day', 'days_inactive']].to_string(index=False))

# -----
# 2. CHANNEL DIVERSITY – Breakdown per User
# -----

channel_pivot = all_transactions.groupby(['user_id', 'channel']).size().unstack(fill_value=0)
channel_pivot.columns.name = None
channel_pivot = channel_pivot.reset_index()

# Proporsi per channel
for ch in ['Call', 'Email', 'SMS', 'WhatsApp']:
```

```

if ch in channel_pivot.columns:
    total = channel_pivot[['Call', 'Email', 'SMS', 'WhatsApp']].sum(axis=1)
    channel_pivot[f'{ch}_pct'] = (channel_pivot[ch] / total * 100).round(1)

print("\n=== Channel Mix per User ===")
print(channel_pivot.to_string(index=False))

# -----
# 3. RFM ANALYSIS – Monetary = total_revenue
# -----

# Hitung revenue per user
rev_call = dfs['Call.csv'].copy(); rev_call['revenue'] = rev_call.apply(lambda x: x['total_price'] if x['is_charge'] else 0,
rev_email = dfs['Email.csv'].copy(); rev_email['revenue'] = rev_email.apply(lambda x: x['price'] if x['is_charge'] else 0, axis
rev_sms = dfs['Sms.csv'].copy(); rev_sms['revenue'] = rev_sms.apply(lambda x: x['total_price'] if x['is_charge'] else 0,
rev_wa = dfs['Whatsapp.csv'].copy(); rev_wa['revenue'] = rev_wa.apply(lambda x: x['price'] if x['is_charge'] else 0, axis=1

user_revenue = pd.concat([
    rev_call[['user_id', 'revenue']],
    rev_email[['user_id', 'revenue']],
    rev_sms[['user_id', 'revenue']],
    rev_wa[['user_id', 'revenue']]
]).groupby('user_id')['revenue'].sum().reset_index()
user_revenue.columns = ['user_id', 'total_revenue']

# RFM
rfm = user_profile[['user_id', 'days_inactive', 'active_days']].copy()
rfm.columns = ['user_id', 'Recency', 'Frequency']
rfm = rfm.merge(user_revenue, on='user_id')
rfm.rename(columns={'total_revenue': 'Monetary'}, inplace=True)

rfm['R_score'] = 4
rfm['F_score'] = pd.qcut(rfm['Frequency'].rank(method='first'), q=4, labels=[1,2,3,4]).astype(int)
rfm['M_score'] = pd.qcut(rfm['Monetary'].rank(method='first'), q=4, labels=[1,2,3,4]).astype(int)

rfm['RFM_score'] = rfm['R_score'].astype(str) + rfm['F_score'].astype(str) + rfm['M_score'].astype(str)
rfm['RFM_total'] = rfm['R_score'] + rfm['F_score'] + rfm['M_score']

def rfm_segment(row):
    f, m = row['F_score'], row['M_score']
    if f >= 3 and m >= 3: return 'Champions'
    elif f >= 3 and m == 2: return 'Loyal Clients'
    elif f == 2 and m >= 3: return 'Potential Loyalist'
    elif f <= 2 and m <= 2: return 'Needs Attention'
    else: return 'Average'

rfm['Segment'] = rfm.apply(rfm_segment, axis=1)

print("=== RFM Segmentation (Monetary = Revenue) ===")
print(rfm[['user_id', 'Recency', 'Frequency', 'Monetary', 'F_score', 'M_score', 'RFM_total', 'Segment']]\
    .sort_values('RFM_total', ascending=False).to_string(index=False))

print("\n=== Distribusi Segment ===")
print(rfm['Segment'].value_counts())

# -----
# 4. RULE-BASED SEGMENTATION – fixed
# -----

pct_cols = [f'{c}_pct' for c in ['Call', 'Email', 'SMS', 'WhatsApp'] if f'{c}_pct' in channel_pivot.columns]

seg = user_profile.merge(rfm[['user_id', 'F_score', 'M_score', 'RFM_total', 'Segment']], on='user_id')
seg = seg.merge(channel_pivot[['user_id'] + pct_cols], on='user_id')
seg = seg.merge(dfs['Users.csv'][['user_id', 'user_name', 'field_of_business']], on='user_id')

vol_median = seg['total_messages'].median()
freq_median = seg['active_days'].median()

def rule_segment(row):
    vol = row['total_messages']
    freq = row['active_days']
    div = row['channel_count']
    wa = row.get('WhatsApp_pct', 0)

    if vol > vol_median and freq >= 91 and div == 4:
        return 'Power User'
    elif vol > vol_median and div <= 2:

```

```

        return 'High Volume, Low Diversity'
    elif vol <= vol_median and freq >= 91:
        return 'Frequent, Low Volume'
    elif wa > 60:
        return 'WhatsApp Dependent'
    else:
        return 'Average User'

seg['Rule_Segment'] = seg.apply(rule_segment, axis=1)

print("=== Rule-Based Segmentation ===")
print(seg[['user_id', 'user_name', 'field_of_business', 'total_messages', 'active_days', 'channel_count', 'Rule_Segment']]\
      .sort_values('total_messages', ascending=False).to_string(index=False))

print("\n=== Distribusi Rule Segment ===")
print(seg['Rule_Segment'].value_counts())

```

```

=== User Profile Overview ===

```

user_id	total_messages	active_days	channel_count	avg_msg_per_day	days_inactive
user_1	24496	91	4	269.2	0
user_10	1525	91	4	16.8	0
user_11	1559	90	4	17.3	0
user_12	1487	91	4	16.3	0
user_13	1530	90	4	17.0	0
user_14	1523	90	4	16.9	0
user_15	1488	90	4	16.5	0
user_16	1593	91	4	17.5	0
user_17	1510	90	4	16.8	0
user_18	1573	91	4	17.3	0
user_19	1552	90	4	17.2	0
user_2	24518	91	4	269.4	0
user_20	1554	91	4	17.1	0
user_3	24635	91	4	270.7	0
user_4	6230	91	4	68.5	0
user_5	6198	91	4	68.1	0
user_6	6010	91	4	66.0	0
user_7	6132	91	4	67.4	0
user_8	6117	91	4	67.2	0
user_9	1519	91	4	16.7	0

```

=== Channel Mix per User ===

```

user_id	Call	Email	SMS	WhatsApp	Call_pct	Email_pct	SMS_pct	WhatsApp_pct
user_1	2561	7938	3960	10037	10.5	32.4	16.2	41.0
user_10	161	492	259	613	10.6	32.3	17.0	40.2
user_11	160	500	260	639	10.3	32.1	16.7	41.0
user_12	166	499	234	588	11.2	33.6	15.7	39.5
user_13	168	480	257	625	11.0	31.4	16.8	40.8
user_14	161	516	226	620	10.6	33.9	14.8	40.7
user_15	151	468	251	618	10.1	31.5	16.9	41.5
user_16	156	530	263	644	9.8	33.3	16.5	40.4
user_17	161	473	230	646	10.7	31.3	15.2	42.8
user_18	165	509	270	629	10.5	32.4	17.2	40.0
user_19	157	493	246	656	10.1	31.8	15.9	42.3
user_2	2502	8107	3989	9920	10.2	33.1	16.3	40.5
user_20	154	504	260	636	9.9	32.4	16.7	40.9
user_3	2586	7955	4051	10043	10.5	32.3	16.4	40.8
user_4	639	2105	993	2493	10.3	33.8	15.9	40.0
user_5	639	2040	1022	2497	10.3	32.9	16.5	40.3
user_6	658	1921	962	2469	10.9	32.0	16.0	41.1
user_7	648	1945	1017	2522	10.6	31.7	16.6	41.1
user_8	603	1989	1006	2519	9.9	32.5	16.4	41.2
user_9	153	536	244	586	10.1	35.3	16.1	38.6

```

=== RFM Segmentation (Monetary = Revenue) ===

```

user_id	Recency	Frequency	Monetary	F_score	M_score	RFM_total	Segment
user_5	0	91	1310570	4	4	12	Champions
user_7	0	91	1327690	4	4	12	Champions
user_6	0	91	1294970	4	3	11	Champions
user_2	0	91	5198910	3	4	11	Champions
user_3	0	91	5207360	3	4	11	Champions
user_8	0	91	1284620	4	3	11	Champions
user_20	0	91	342240	3	3	10	Champions
user_1	0	91	5125530	2	4	10	Potential Loyalist
user_4	0	91	1276070	3	3	10	Champions
user_18	0	91	344080	3	3	10	Champions
user_9	0	91	317510	4	1	9	Average

- ✦ Segmentation (Penentuan Pemilihan dan Alasannya)
- ✦ Client Segments (RFM Framework or Other Clustering Framework)

Segmentasi pada project ini tidak bisa dilakukan hanya semata-mata menggunakan RFM Framework, yakni:

- R(Recency) - Menunjukkan kapan user-user tersebut terakhir kali aktif / berapa hari sejak terakhir aktif mengirim pesan (last_active_days/last_message_date). Alasannya, diketahui bahwa semua user aktif selama periode data (Recency=0-1 hari (ZERO-VARIANCE) / semua 20 klien aktif di tanggal terakhir data, sehingga recency_days = 0-1 hari untuk setiap user, tidak ada user/client oca yang inaktif lebih dari 14 hari). Oleh karena itu, untuk segmentasi dengan metrik "Recency/last active days / days inactive" ini tidak dapat dipakai atau tidak bisa menjadi pembeda antar segmen nya. Tidak membedakan apapun.
- F(Frequency) - Kalau frequency yang dimaksud disini menunjukkan active days (Berapa kali/hari user aktif dalam periode / total tiap-tiap user dari keseluruhan itu berapa hari aktifnya), ini juga tidak bisa dijadikan pembeda (differentiator) antar segmen nya. Karena diketahui dari data, tiap user itu aktif tiap harinya (full/penuh melakukan transaksi atau mengirim pesan). Active Days nya = 90-91 hari dari 91 hari, std deviasinya hanya 0.47 (tidak bisa menjadi pembeda yang baik, **Practically identical untuk semua klien**). Kecuali, Frequency yang dimaksud disini kita ganti ke Average Message per Day nya / Rata-rata pesan yang dikirim per harinya (breakdown per user).
- M(Monetary) - Ini satu-satunya yang bisa menjadi pembeda (differentiator) antar segmen untuk melakukan segmentasi, yaitu menggunakan variabel/atribut/metrik total_revenue yang dihasilkan per user. Revenue yang dimaksud disini dihitung apabila pesan yang dikirimkan user/client berhasil terkirim (is_charge=TRUE). Monetary pada kasus ini lebih ke volume pesan yang dikirim per user nya (message volume) dan revenue nya.

Oleh karena perbedaan paling mendasar/paling terlihat itu di Volume Pesan dan Total Revenue yang Dihasilkan per User nya, segmentasi bisa dilakukan dengan fokus ke membedakan segmen user berdasarkan volume nya (Volume-Tier) dan revenue nya (Revenue-Tier).

Core Metrics (Total_Message_Volume + Total_Revenue)

1. Total Transaction (semua channel) / Total_Message_Volume per User
2. Average_Message_per_Day / Rata-rata pesan yang dikirimkan per harinya (untuk setiap user) : total_volume/total_hari
3. Total Revenue dari semua channel : sum(revenue) per user
4. Revenue per Transaction : (total_revenue/total_volume)

Note: Total TRANSAKSI = Volume Pesan (Message)

Feature Selection

Feature 1: total_txn / total_volume — Total Transaksi Semua Channel per User/Klien

- Cara Menghitung: COUNT(transaction_id) dari unified/union dataframe (semua channel, Jan-Mar 2025) GROUP BY / breakdown per user_id.
- Nilai di data: Min 1.487 (user_12 : CV Mart Indo-Retail) → Max 24.635 (user_3 : CV Wisata Indo-Travel).
- Alasan dipilih: Ini adalah differentiator terbesar dan paling konsisten. Dari 20 klien, terlihat 3 natural tier yang sangat jelas secara visual: ~24K (3 klien), ~6K (5 klien), ~1.5K (12 klien). Tanpa feature ini, clustering tidak bisa menangkap perbedaan business value yang paling fundamental.
- Menjawab deep question: "Which segment is most profitable?" — Power User dengan ~24K txn berkontribusi ~60% revenue.
- Kenapa tidak diganti dengan total_revenue: Total_txn dan total_revenue berkorelasi tinggi tapi tidak identik. Klien dengan volume sama bisa punya revenue berbeda tergantung channel mix (SMS menghasilkan Rp591/txn vs Email Rp49/txn). Memasukkan keduanya menangkap kedua dimensi ini.

Feature 2: total_revenue — Total Revenue yang Dihasilkan per User/Klien

- Cara hitung: SUM(revenue) per user_id, semua channel (SUM(price) untuk WA/Email, SUM(total_price) untuk SMS/Call, GROUP BY user_id).
- Nilai di data: Renge di data -> Min Rp300-301 ribu (user_12 : CV Mart Indo-Retail) → Max Rp5,2 Juta (user_3 : CV Wisata Indo-Travel). Bervariasi seiring volume tapi tidak proporsional sempurna, karena pricing berbeda per channel (Revenue per Message -> WA: Rp205,16/tx, SMS: Rp590,65/tx (paling mahal/tinggi), Email: Rp39,28/tx (paling murah/rendah), Call: Rp180,74/tx).
- Alasan dipilih: Revenue adalah metric yang langsung relevan untuk OCA Blast sebagai bisnis. Dua klien dengan volume hampir sama bisa punya revenue berbeda jika channel mix-nya berbeda. Misalnya klien yang lebih banyak pakai SMS (Rp591/txn) vs Email (Rp39/txn) menghasilkan revenue 14x lebih tinggi per transaksi. Feature ini menangkap dimensi business value yang tidak tertangkap oleh volume saja. Dipakai sebagai profiling, bukan pembentuk tier (karena sangat berkorelasi dengan volume). Mengkuantifikasi nilai

bisnis per klien dalam Rupiah — lebih mudah dikomunikasikan ke stakeholder non-teknis dibanding volume. Enterprise 59.8% revenue dari 15% klien = justifikasi business case untuk investasi CSM dedicated.

- Menjawab deep question: "Which segment is most profitable?" — lengkap ketika dikombinasikan dengan total_txn.

Feature 3: avg_message_per_day — Rata-rata Transaksi/Volume Pesan yang dikirim per Hari

- Cara hitung: $\text{total_txn} / \text{active_days per user_id}$ ($\text{total_volume} / \text{COUNT}(\text{DISTINCT DATE}(\text{created_at}))$ per user)
- Nilai di data: Berkisar dari ~16–17 txn/hari (Regular) sampai ~270 txn/hari (Power User). Std Dev signifikan. Range di data: 16.3/hari (CV Mart Indo) hingga 270.7/hari (CV Wisata Indo)
- Alasan dipilih: Mengukur intensitas penggunaan harian dimensi yang tidak tertangkap oleh total_txn saja. Ini penting karena dua klien dengan total transaksi sama bisa punya pola yang sangat berbeda: satu klien blast besar tapi tidak menentu/kadang-kadang, klien lain blast kecil tapi konsisten setiap hari. Untuk scheduling optimization dan capacity planning OCA Blast, intensitas harian sangat relevan. CV (Coefficient of Variance) 133% — konfirmasi independen untuk tier. Jika total_volume tinggi karena periode aktif lebih lama, avg_msg_per_day akan rendah, ini mendeteksi perbedaan itu. Enterprise 270/hari = operasional bergantung penuh pada platform (blast menjadi core business process). Standard 17/hari = penggunaan casual/occasional. Perbedaan 16x ini berarti kebutuhan infrastruktur dan support benar-benar berbeda.
- Menjawab deep question: "Do high-volume users show unique hourly patterns?" — Power User dengan avg_txn/day ~270 menunjukkan peak jam 17–18 WIB, berbeda dengan Growth/Regular yang peak 18–22 WIB.
- Catatan: Meskipun berkorelasi dengan total_txn (karena active_days seragam = 90–91), variabel ini tetap dipertahankan karena memberi interpretasi berbeda: bukan "berapa banyak total" tapi "seberapa intensif setiap harinya."

Feature 4: revenue_per_txn — Revenue per Transaksi (Efisiensi Monetisasi)

- Cara hitung: $\text{total_revenue} / \text{total_txn per user_id}$.
- Nilai di data: Berkisar Rp200–Rp220 per transaksi. Terlihat kecil, tapi variasinya bermakna secara bisnis.
- Alasan dipilih: Ini adalah satu-satunya feature yang menangkap channel mix behavior secara implisit. Meskipun semua klien pakai 4 channel, proporsi penggunaan sedikit berbeda. Klien yang lebih banyak pakai SMS (Rp591/txn) akan punya revenue_per_txn lebih tinggi dibanding klien yang dominan Email (Rp49/txn). Feature ini justru yang membedakan Growth User dari Regular User di clustering nya, dua grup yang secara volume tidak terlalu jauh, tapi punya profil monetisasi berbeda.
- Menjawab deep question: "Do mono-channel users have upsell potential?" — Klien dengan revenue_per_txn rendah mengindikasikan channel mix yang didominasi Email, sinyal untuk upsell ke SMS/WA.

decline_pct — Penurunan volume Jan→Mar (monthly trend)

- Cara hitung: $(\text{baseline_avg} - \text{vol_Mar}) / \text{baseline_avg} \times 100$, di mana $\text{baseline_avg} = (\text{vol_Jan} + \text{vol_Feb}) / 2$
- Range di data: 10.0% (user_18 - CV Jalan -Hospitality) hingga 29.8% (user_11 - PT Travel Nusantara - Hospitality)
- Alasan dipilih sebagai Layer 2 (bukan fitur clustering): Decline bersifat temporal — bisa berubah bulan depan. Jika dimasukkan ke clustering, segmen akan tidak stabil (klien yang sedang dalam campaign besar bulan Jan akan terlihat "declining" di Feb, padahal normal). Dipakai sebagai enrichment setelah tier terbentuk: tier menunjukkan "siapa kliennya" (permanen), decline menunjukkan "kondisi klien saat ini" (dinamis).

Volume-Tier Segmentation

```
# -----
# 5. VOLUME TIER SEGMENTATION
# -----
# RULED BASED SEGMENTATION
def volume_tier(vol):
    if vol >= 20000:
        return 'Tier 1 - Enterprise'
    elif vol >= 5000:
        return 'Tier 2 - Growth'
    else:
        return 'Tier 3 - Standard'

seg['Volume_Tier'] = seg['total_messages'].apply(volume_tier)

print("=== Volume Tier ===")
print(seg[['user_id', 'user_name', 'field_of_business', 'total_messages', 'avg_msg_per_day', 'Volume_Tier']]\
      .sort_values('total_messages', ascending=False).to_string(index=False))

print("\n=== Statistik per Tier ===")
tier_stats = seg.groupby('Volume_Tier').agg(
```



```

    jumlah_klien    = ('user_id', 'count'),
    avg_messages    = ('total_messages', 'mean'),
    total_messages  = ('total_messages', 'sum'),
    avg_msg_per_day = ('avg_msg_per_day', 'mean'),
).round(1)
print(tier_stats)

# -----
# 6. BEHAVIORAL PATTERN – Pola Waktu per Tier
# -----

all_transactions['hour'] = all_transactions['created_at'].dt.hour
all_transactions['weekday'] = all_transactions['created_at'].dt.day_name()
all_transactions['month'] = all_transactions['created_at'].dt.to_period('M')

# Gabung tier ke all_transactions
tier_map = seg[['user_id', 'Volume_Tier']].set_index('user_id')['Volume_Tier']
all_transactions['Volume_Tier'] = all_transactions['user_id'].map(tier_map)

# Peak hour per tier
print("\n=== Peak Hour per Tier ===")
hour_tier = all_transactions.groupby(['Volume_Tier', 'hour']).size().reset_index(name='count')
for tier in ['Tier 1 – Enterprise', 'Tier 2 – Growth', 'Tier 3 – Standard']:
    sub = hour_tier[hour_tier['Volume_Tier']==tier].sort_values('count', ascending=False).head(3)
    print(f"\n{tier}:")
    print(sub.to_string(index=False))

# Tren bulanan per tier
print("\n=== Tren Bulanan per Tier ===")
monthly_tier = all_transactions.groupby(['Volume_Tier', 'month']).size().reset_index(name='count')
print(monthly_tier.pivot(index='month', columns='Volume_Tier', values='count').to_string())

# -----
# 7. UPSELL POTENTIAL
# -----

print("\n=== Upsell Potential (Tier 2 & 3 dengan channel_count < 4) ===")
upsell = seg[(seg['channel_count'] < 4) | (seg['Volume_Tier'] != 'Tier 1 – Enterprise')]
print(upsell[['user_id', 'user_name', 'field_of_business', 'total_messages', 'channel_count', 'Volume_Tier']]
        .sort_values('total_messages', ascending=False).to_string(index=False))

# -----
# 8. FINAL SEGMENTATION TABLE
# -----

final = seg[['user_id', 'user_name', 'field_of_business', 'total_messages',
             'active_days', 'avg_msg_per_day', 'channel_count',
             'WhatsApp_pct', 'Email_pct', 'SMS_pct', 'Call_pct',
             'Segment', 'Volume_Tier']].sort_values('total_messages', ascending=False)

print("\n=== Final Segmentation Table ===")
print(final.to_string(index=False))

=== Volume Tier ===
user_id      user_name  field_of_business  total_messages  avg_msg_per_day  Volume_Tier
user_3      CV Wisata Indo      Travel      24635      270.7  Tier 1 – Enterprise
user_2      PT Dana Nusantara      Finance      24518      269.4  Tier 1 – Enterprise
user_1      CV Land Sejahtera      Real Estate      24496      269.2  Tier 1 – Enterprise
user_4      PT Konektivitas Indo  Telecommunications      6230      68.5    Tier 2 – Growth
user_5      PT Karya Sejahtera      Education      6198      68.1    Tier 2 – Growth
user_7      PT Retail Sejahtera      Retail      6132      67.4    Tier 2 – Growth
user_8      PT Belajar Nusantara      Education      6117      67.2    Tier 2 – Growth
user_6      PT Logistik Mandiri      Logistics      6010      66.0    Tier 2 – Growth
user_16     PT Estate Sejahtera      Real Estate      1593      17.5    Tier 3 – Standard
user_18     CV Jalan Mandiri      Travel      1573      17.3    Tier 3 – Standard
user_11     PT Travel Nusantara      Hospitality      1559      17.3    Tier 3 – Standard
user_20     CV Hotel Jaya      Hospitality      1554      17.1    Tier 3 – Standard
user_19     CV Niaga Indo      eCommerce      1552      17.2    Tier 3 – Standard
user_13     PT Saham Global      Finance      1530      17.0    Tier 3 – Standard
user_10     PT Mart Jaya      Retail      1525      16.8    Tier 3 – Standard
user_14     CV Hotel Nusantara      Hospitality      1523      16.9    Tier 3 – Standard
user_9      PT Citra Global      Hospitality      1519      16.7    Tier 3 – Standard
user_17     PT Karya Jaya      Education      1510      16.8    Tier 3 – Standard
user_15     PT Edu Mandiri      Education      1488      16.5    Tier 3 – Standard
user_12     CV Mart Indo      Retail      1487      16.3    Tier 3 – Standard

=== Statistik per Tier ===
                jumlah_klien  avg_messages  total_messages  \
Volume_Tier

```



```
Tier 1 – Enterprise      3      24549.7      73649
Tier 2 – Growth          5      6137.4      30687
Tier 3 – Standard       12     1534.4      18413
```

```
                                avg_msg_per_day
Volume_Tier
Tier 1 – Enterprise      269.8
Tier 2 – Growth          67.4
Tier 3 – Standard        17.0
```

```
=== Peak Hour per Tier ===
```

```
Tier 1 – Enterprise:
      Volume_Tier  hour  count
Tier 1 – Enterprise    18   5993
Tier 1 – Enterprise    17   5947
Tier 1 – Enterprise    16   5935
```

```
Tier 2 – Growth:
      Volume_Tier  hour  count
Tier 2 – Growth    21   2514
Tier 2 – Growth     0   2446
Tier 2 – Growth    18   2445
```

```
Tier 3 – Standard:
      Volume_Tier  hour  count
Tier 3 – Standard    19   1537
Tier 3 – Standard    23   1517
Tier 3 – Standard    22   1490
```

```
--- Tren Bulanan per Tier ---
```

Revenue-Tier Segmentation

```
# -----
# 9. REVENUE PER TIER
# -----

# Gabung revenue dari masing-masing channel
rev_wa = dfs['Whatsapp.csv'][['user_id', 'is_charge', 'price']].copy()
rev_wa['revenue'] = rev_wa.apply(lambda x: x['price'] if x['is_charge'] else 0, axis=1)

rev_sms = dfs['Sms.csv'][['user_id', 'is_charge', 'total_price']].copy()
rev_sms['revenue'] = rev_sms.apply(lambda x: x['total_price'] if x['is_charge'] else 0, axis=1)

rev_email = dfs['Email.csv'][['user_id', 'is_charge', 'price']].copy()
rev_email['revenue'] = rev_email.apply(lambda x: x['price'] if x['is_charge'] else 0, axis=1)

rev_call = dfs['Call.csv'][['user_id', 'is_charge', 'total_price']].copy()
rev_call['revenue'] = rev_call.apply(lambda x: x['total_price'] if x['is_charge'] else 0, axis=1)

all_rev = pd.concat([
    rev_wa[['user_id', 'revenue']],
    rev_sms[['user_id', 'revenue']],
    rev_email[['user_id', 'revenue']],
    rev_call[['user_id', 'revenue']]
])

user_revenue = all_rev.groupby('user_id')['revenue'].sum().reset_index()
user_revenue.columns = ['user_id', 'total_revenue']

seg = seg.merge(user_revenue, on='user_id')

print("=== Revenue per Tier ===")
tier_rev = seg.groupby('Volume_Tier').agg(
    jumlah_klien = ('user_id', 'count'),
    total_revenue = ('total_revenue', 'sum'),
    avg_revenue = ('total_revenue', 'mean'),
).round(0)
tier_rev['revenue_share_%'] = (tier_rev['total_revenue'] / tier_rev['total_revenue'].sum() * 100).round(1)
print(tier_rev)

# -----
# 10. GABUNG CHURN RISK
# -----

churn_map = churn2[['user_id', 'decline_pct', 'risk_level']].copy()
seg = seg.merge(churn_map, on='user_id', how='left')
```

```

print("\n=== Churn Risk per Tier ===")
print(seg.groupby(['Volume_Tier', 'risk_level'])['user_id'].count().reset_index().to_string(index=False))

# -----
# 11. FINAL TABLE + EXPORT
# -----

final_seg = seg[[
    'user_id', 'user_name', 'field_of_business',
    'Volume_Tier', 'Segment',
    'total_messages', 'avg_msg_per_day', 'active_days',
    'total_revenue',
    'WhatsApp_pct', 'Email_pct', 'SMS_pct', 'Call_pct',
    'decline_pct', 'risk_level'
]].sort_values('total_messages', ascending=False)

print("\n=== Final Segmentation Table ===")
print(final_seg.to_string(index=False))

OUTPUT_PATH = r"F:\5. Project\Jazmeen\RevouXTelkom\data\cleaned"
final_seg.to_csv(f"{OUTPUT_PATH}\\user_segmentation.csv", index=False)
print("\nSaved: user_segmentation.csv")

```

```

=== Revenue per Tier ===

```

Volume_Tier	jumlah_klien	total_revenue	avg_revenue	revenue_share_%
Tier 1 – Enterprise	3	15531800	5177267.0	59.9
Tier 2 – Growth	5	6493920	1298784.0	25.0
Tier 3 – Standard	12	3920700	326725.0	15.1

```

=== Churn Risk per Tier ===

```

Volume_Tier	risk_level	user_id
Tier 1 – Enterprise	Stable	3
Tier 2 – Growth	Stable	5
Tier 3 – Standard	Medium	5
Tier 3 – Standard	Stable	7

```

=== Final Segmentation Table ===

```

user_id	user_name	field_of_business	Volume_Tier	Segment	total_messages	avg_msg_per_day	active_
user_3	CV Wisata Indo	Travel	Tier 1 – Enterprise	Champions	24635	270.7	
user_2	PT Dana Nusantara	Finance	Tier 1 – Enterprise	Champions	24518	269.4	
user_1	CV Land Sejahtera	Real Estate	Tier 1 – Enterprise	Potential Loyalist	24496	269.2	
user_4	PT Konektivitas Indo	Telecommunications	Tier 2 – Growth	Champions	6230	68.5	
user_5	PT Karya Sejahtera	Education	Tier 2 – Growth	Champions	6198	68.1	
user_7	PT Retail Sejahtera	Retail	Tier 2 – Growth	Champions	6132	67.4	
user_8	PT Belajar Nusantara	Education	Tier 2 – Growth	Champions	6117	67.2	
user_6	PT Logistik Mandiri	Logistics	Tier 2 – Growth	Champions	6010	66.0	
user_16	PT Estate Sejahtera	Real Estate	Tier 3 – Standard	Needs Attention	1593	17.5	
user_18	CV Jalan Mandiri	Travel	Tier 3 – Standard	Champions	1573	17.3	
user_11	PT Travel Nusantara	Hospitality	Tier 3 – Standard	Needs Attention	1559	17.3	
user_20	CV Hotel Jaya	Hospitality	Tier 3 – Standard	Champions	1554	17.1	
user_19	CV Niaga Indo	eCommerce	Tier 3 – Standard	Needs Attention	1552	17.2	
user_13	PT Saham Global	Finance	Tier 3 – Standard	Needs Attention	1530	17.0	
user_10	PT Mart Jaya	Retail	Tier 3 – Standard	Needs Attention	1525	16.8	
user_14	CV Hotel Nusantara	Hospitality	Tier 3 – Standard	Needs Attention	1523	16.9	
user_9	PT Citra Global	Hospitality	Tier 3 – Standard	Average	1519	16.7	
user_17	PT Karya Jaya	Education	Tier 3 – Standard	Needs Attention	1510	16.8	
user_15	PT Edu Mandiri	Education	Tier 3 – Standard	Needs Attention	1488	16.5	
user_12	CV Mart Indo	Retail	Tier 3 – Standard	Needs Attention	1487	16.3	

Saved: user_segmentation.csv

Segmentasi ini langsung mendukung keputusan bisnis: (1) Alokasi CSM ke klien Enterprise yang menghasilkan 59.8% revenue. (2) Prioritas outreach churn prevention ke 5 klien dengan High/Medium Risk. (3) Strategi upsell yang berbeda: Growth didorong naik volume, Standard didorong adopsi Scheduled Blast. Hasilnya langsung dapat digunakan oleh Account Manager dan tim Sales OCA Blast.

Metode Clustering

Pada Project 2 ini, kita masuk ke tahap Membuat Segmentasi User untuk OCA Blast.

Segmentasi Customer adalah pengelompokan customer berdasarkan karakteristik / ciri-ciri/ profile mereka (yang dilihat dari variabel/data). Tujuannya adalah untuk bisa memahami ciri-ciri dari setiap customer dan juga bisa membuat personalized treatment (treatment/perlakuan yang dipersonalisasi), disesuaikan dengan preferensi & karakteristik tiap kelompok customer, sehingga treatment yang kita lakukan bisa berjalan lebih efektif.

Kita perlu melakukan segmentasi customer untuk mencapai tujuan bisnis dengan memberikan **personalized treatment** yang **sesuai dengan profile customer** (setiap customer memiliki profile yang berbeda-beda), sehingga nantinya kita dapat memberikan treatment (perlakuan) yang cocok untuk tiap-tiap kelompok customer berdasarkan karakteristik/kemiripan/ciri-cirinya itu seperti apa.

▼ K-Means Clustering

Pada project 2 Virtual Internship Program RevoU x Telkom Indonesia untuk studi kasus pada OCA Blast ini, saya memilih untuk menggunakan dan mencoba melakukan segmentasi user (20 klien aktif OCA) menggunakan metode clustering K-Means.

K-Means disini memungkinkan kita untuk menggunakan serta menggabungkan beberapa atribut/variabel yang dapat menggabungkan variabel penggunaan platform untuk 20 klien aktif OCA. Pertimbangan juga dilihat dari apa yang kita punya dari datasetnya dengan segala kondisi datanya dan kapasitas datanya.

Dengan K-Means, kita bisa memasukkan dimensi skala penggunaan (total_volume), nilai bisnis (total_revenue), intensitas penggunaan/pengiriman pesan harian (avg_msg_per_day), dan volume decline percentage nya untuk tiap user secara bersamaan untuk menghasilkan user personas yang benar-benar merepresentasikan perilaku klien B2B secara menyeluruh, serta setelah dicoba sebelumnya menggunakan RFM Framework, hasilnya tidak dapat membedakan antar segmen dengan baik, salah satu alasannya dikarenakan variabel recency yang hampir sama antar klien. Jadi, disini tetap eksplor dengan memaksimalkan yang bisa menjadi variabel pembedanya (diantaranya yang paling / sangat terlihat dari volume dan revenue nya, serta decline percentage untuk early churn risk signal)

- K-Means memungkinkan kita menemukan kelompok klien yang secara volume hampir sama/mirip, tapi secara efisiensi monetisasi sangat berbeda. Misalnya di data OCA Blast: user_4 (PT Konektivitas Indo, 6.230 tx, rev/tx Rp 219) dan user_5 (PT Karya Sejahtera, 6.198 tx, rev/tx Rp 226) punya volume (total transaksi) yang hampir sama tetapi jika ditelusuri kembali efisiensi channel mix-nya berbeda karena proporsi SMS vs Email yang berbeda. K-Means mendeteksi perbedaan ini dan menempatkan mereka di cluster yang berbeda, sementara ranking volume biasa akan menempatkan mereka di tier yang sama. Tanpa K-Means: kedua klien ini masuk "tier yang sama" hanya karena volume mirip. Dengan K-Means: cluster algorithm melihat kombinasi volume + revenue + intensity secara bersamaan → pengelompokan lebih tepat.
- avg_msg_per_day menangkap operational dependency yang tidak tergambar dari total volume saja. Dua klien bisa punya total volume sama selama 90 hari, tapi dengan intensitas yang sangat berbeda: satu klien blast besar 1–2 kali sebulan, klien lain blast kecil setiap hari. Keduanya punya implikasi operasional yang berbeda bagi OCA Blast. K-Means dengan avg_msg_per_day memungkinkan untuk kita memisahkan dua pola ini.

```
# Import tambahan untuk K-Means
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.ticker as mticker
import seaborn as sns
# Pengaturan untuk menghilangkan warning (biar notebook bersih saja, optional)
import warnings
warnings.filterwarnings('ignore')
```

```
# =====
# PROJECT 2 – ACTIVE USER BEHAVIOR & K-MEANS SEGMENTATION
# OCA Blast | RevoU x Telkom Indonesia Virtual Internship
# Disclaimer: "The data provided is a representation of the original data
# which amount has been adjusted. The data used is NOT representation of
# original data, adjusted for educational purposes and does not reflect
# the actual business condition of the company."
# =====
```

▼ Preparation Dataframe untuk User Segmentation

Di tahap ini kita memilih dan memasukkan variabel/atribut (features) apa saja yang ingin digunakan untuk segmentasi/clustering.

Langkah ini dilakukan untuk membuat dataframe baru (tabel baru) yang isinya variabel-variabel atau kolom-kolom yang diperlukan dalam membuat segmentasi user/klien. Pertama, kita perlu mengidentifikasi kolom-kolom mana saja yang sekiranya relevan untuk menjawab tujuan segmentasi, yaitu mengelompokkan nasabah-nasabah berdasarkan pola perilaku transaksi mereka dalam melakukan pengiriman pesan dari Jan-Mar 2025 dan mengidentifikasi karakteristik tiap segmen serta menentukan segmen mana yang perlu diprioritaskan.

OCA memiliki 20 klien aktif dari beberapa industri berbeda

```
# Feature columns untuk clustering
CLUSTERING_FEATURES = [
    'total_revenue',    # business value
    'total_messages',   # skala aktivitas
]
```

```
# Langkah 2 : Feature scaling - StandardScaler
# Langkah ini dilakukan untuk menormalisasikan data, karena K-Means menggunakan Euclidean distance. Tanpa normalisasi:
# - total_volume (range 1.487-24.635) akan mendominasi jarak
# StandardScaler: mean=0, std=1 → setiap feature berbobot setara
```

```
# CATATAN: decline_pct TIDAK dimasukkan ke clustering features
# Alasan: decline adalah sinyal TEMPORAL (bisa berubah bulan depan)
# Jika dimasukkan, klien yang sedang dalam campaign besar bulan Jan
# akan masuk cluster berbeda bulan depan meski perilaku dasarnya sama
# → Dipakai sebagai post-clustering enrichment (Layer 2) saja
```

```
X = seg[CLUSTERING_FEATURES].copy()
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=CLUSTERING_FEATURES,
                           index=seg.index)
```

```
print("✅ StandardScaler applied")
print("\nSebelum scaling (range asli):")
print(X.describe().round(2).to_string())
print("\nSetelah scaling (mean=0, std=1):")
print(X_scaled_df.describe().round(3).to_string())
```

✅ StandardScaler applied

Sebelum scaling (range asli):

	total_revenue	total_messages
count	20.00	20.00
mean	1297321.00	6137.45
std	1724053.21	8180.00
min	300770.00	1487.00
25%	322485.00	1524.50
50%	341485.00	1566.00
75%	1298870.00	6148.50
max	5207360.00	24635.00

Setelah scaling (mean=0, std=1):

	total_revenue	total_messages
count	20.000	20.000
mean	-0.000	0.000
std	1.026	1.026
min	-0.593	-0.583
25%	-0.580	-0.579
50%	-0.569	-0.573
75%	0.001	0.001
max	2.327	2.320

▼ Determine Number of Clustering (K)

Langkah selanjutnya adalah menentukan jumlah kelompok (cluster) yang paling optimal yang dapat dibuat menggunakan dua alat bantu, yakni:

- Elbow Method
- Silhouette Method/Analysis

Tujuan segmentasi juga perlu dipertimbangkan dalam memilih jumlah kelompok (cluster), cari jumlah cluster yang balance, tidak terlalu banyak dan nilai inertia sudah cukup kecil, tetapi cukup baik untuk membedakan/mengelompokkan user-user OCA

✓ Elbow Method

Disini kita akan menentukan jumlah kelompok (cluster) dengan melihat dan mencari titik sikunya/titik patahannya. Kita akan memilih K ketika nilai-inertia nya **mulai stabil atau mulai landai**. Nilai inertia seperti tingkat kesalahan / error.

- Elbow Method menunjukkan berapa inertia yang dihasilkan ketika kita memilih jumlah cluster sebanyak "N"
- Kita tentu menginginkan nilai inertia yang sekecil mungkin
- Masalahnya adalah semakin banyak jumlah cluster, inertianya sudah pasti makin kecil (kita perlu yang BALANCE, dalam arti jumlah clusternya tidak terlalu banyak, tetapi nilai inertianya sudah cukup kecil)
- Kita tidak menginginkan jumlah cluster yang terlalu banyak. Ada tradeoff
- Dengan demikian, kita akan memilih pada bagian "elbow" melalui metode ini di mana yang dicari number of clusters sekecil mungkin yang memberikan inertia yang kecil. Jika digambarkan di plot, jika turunnya sudah tidak curam / mulai landai / turunnya sudah tidak terlalu signifikan, maka stop pemilihan "K" disana

```
# Variabel yang dipakai
X = seg[['total_messages', 'total_revenue']].copy()

# Normalisasi – wajib untuk K-Means
# karena total_messages (ribuan) dan revenue (jutaan) beda skala
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Cari jumlah cluster optimal pakai Elbow Method
inertias = []
sil_scores = []
K_range = range(2, 8)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, km.labels_))

print("K | Inertia | Silhouette Score")
for i, k in enumerate(K_range):
    print(f"{k} | {inertias[i]:.2f} | {sil_scores[i]:.3f}")
```

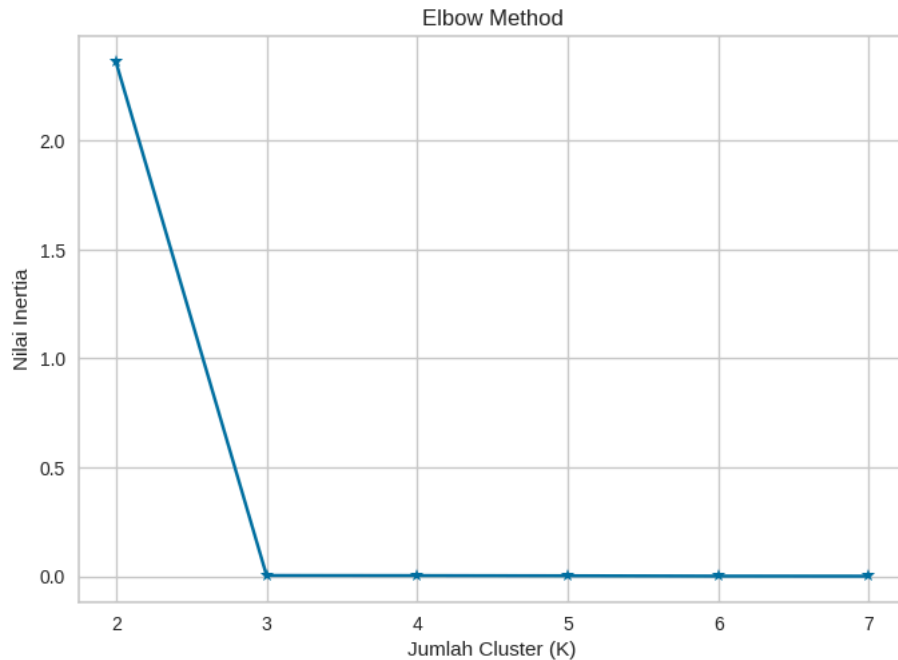
```
K | Inertia | Silhouette Score
2 | 2.36 | 0.912
3 | 0.00 | 0.983
4 | 0.00 | 0.902
5 | 0.00 | 0.701
6 | 0.00 | 0.534
7 | 0.00 | 0.522
```

K=3 memiliki nilai rata-rata silhoutte score yang paling tinggi (0.983) dan menghasilkan nilai inertia yang kecil. Paling mendekati kriteria ideal untuk penentuan jumlah cluster (K=3)

```
# For each n_clusters between 1 and 10, we calculate the distortion value
distortions = []
K = range(2, 8)
for n_clusters in K:
    model = KMeans(n_clusters, random_state=42, n_init='auto')
    model.fit(X_scaled_df)
    distortions.append(model.inertia_)

plt.figure(figsize=(16, 8))
plt.figure()
plt.plot(K, distortions, 'b*-')
plt.xlabel('Jumlah Cluster (K)')
plt.ylabel('Nilai Inertia')
plt.title('Elbow Method')
plt.show()
```

<Figure size 1600x800 with 0 Axes>



Grafik Elbow Method ini memetakan Nilai Inertia (total jarak kuadrat dalam cluster) terhadap Jumlah Cluster (K).

Setelah melihat plotting untuk Elbow Method, dapat dilihat bahwa grafik/plot menunjukkan kurva yang memiliki satu titik "siku" yang sangat tajam / titik transisi atau "patahan" pada K=3. Penurunan drastis terjadi di K=2 ke K=3. Nilai inertia turun tajam dari sekitar 2.36 ke 0,00. Di K=3 adalah titik patahan/titik sikunya, kemudian mulai stabil dan nilai inertia nya sama di 0.00 dari K=3 hingga K=7

Berdasarkan plotting menggunakan elbow method ini, kandidat utama yang saya tangkap untuk jumlah cluster: K=3 (High Impact): Titik siku/patahan terjadi secara jelas di (K=3) setelah itu stabil.

✓ Silhouette Method/Analysis

Pada intinya, metode ini akan memberikan gambaran kepada kita cluster yang kita ingin bentuk BALANCE atau tidak. Silhouette Analysis ini kita akan memilih K dengan memperhatikan tiga kriteria (memilih nilai k yang setidaknya paling mendekati kriteria ideal), yaitu:

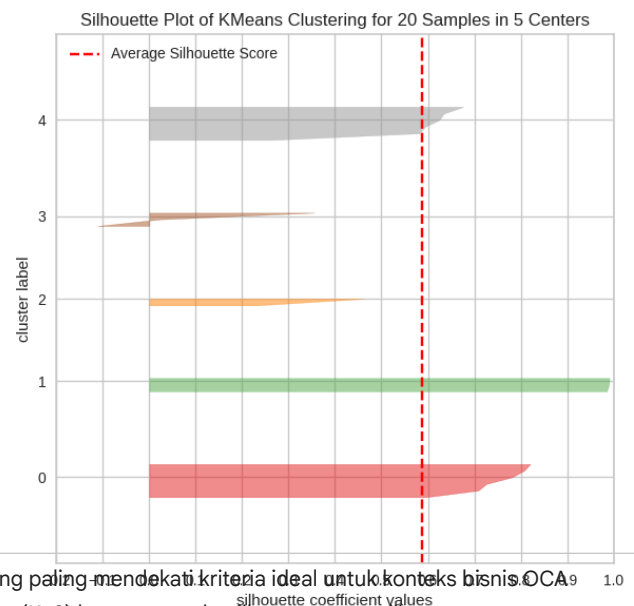
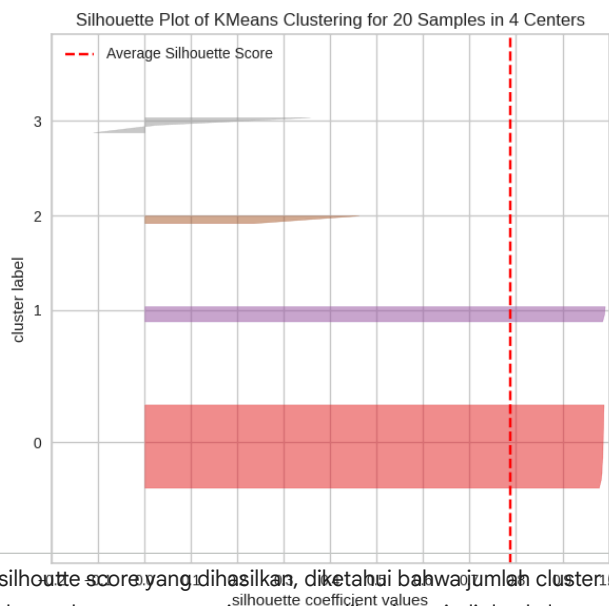
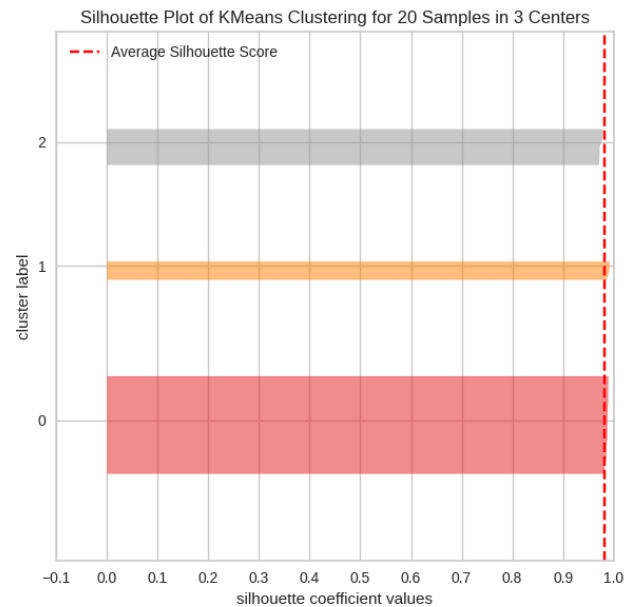
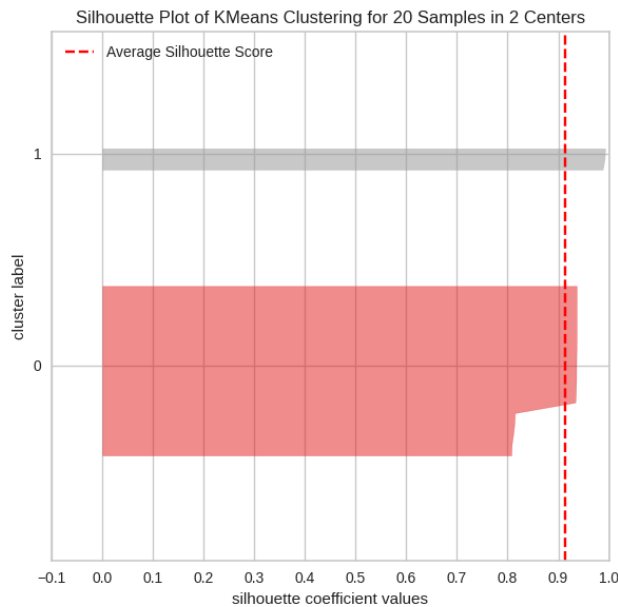
1. Pilih yang silhouette score nya **tinggi**
2. Pilih yang ukuran/proporsi silhouette **cenderung mirip**
3. Pilih yang silhouette **negatifnya sedikit**

```
# Silhouette score plot
K = range(2,8) #[2, 3, 4, 5, 6, 7]
max_K = max(K)
fig, ax = plt.subplots(int(np.ceil(max_K/2)), 2, figsize=(15, 30))

for n_clusters in K:
    model = KMeans(n_clusters, random_state=42, n_init='auto')
    model.fit(X_scaled_df) # untuk meminta model mempelajari/membaca/memahami data segmentasi kita

    q, mod = divmod(n_clusters, 2)
    sil = SilhouetteVisualizer(model, is_fitted=True, ax=ax[q-1][mod])
    sil.fit(X_scaled_df)
    sil.finalize()
    print(f'For k={n_clusters}, the average silhouette score is {sil.silhouette_score}')
```

For k=2, the average silhouette score is 0.9124524687678962
 For k=3, the average silhouette score is 0.982835597531319
 For k=4, the average silhouette score is 0.7877058420045185
 For k=5, the average silhouette score is 0.5870200371007045
 For k=6, the average silhouette score is 0.5060014183588909
 For k=7, the average silhouette score is 0.5219550189425518



Dari silhoutte score yang dihasilkan, diketahui bahwa jumlah cluster (K) yang paling mendekati kriteria ideal untuk konteks bisnis OCA adalah membuat segmentasi user yang dibagi menjadi dua kelompok besar (K=2) karena menghasilkan rata-rata silhoutte score yang paling tinggi yaitu untuk \$k=2\$ adalah 0.9125. Angka ini merupakan yang tertinggi di antara semua opsi (misal: K=3 di angka 0.9828 dan K=4 nilainya turun cukup signifikan ke 0.7877). Skor yang tinggi menunjukkan bahwa setiap objek dalam cluster tersebut sangat mirip satu sama lain dan sangat berbeda dari objek di cluster lain.

Akan tetapi, pertimbangan untuk tidak memilih K=2, karena dirasa jika membagi hanya dalam dua kelompok saja, dirasa tidak terlalu berguna bagi perusahaan. Hal ini diakibatkan dengan kita hanya membagi menjadi dua kelompok/cluster besar, datanya cenderung memiliki segmentasi yang 'dangkal' karena hanya terbagi misal hanya "Power User" vs "Potential Usee". Hal ini memang bisa dianggap "sia-sia" bagi bisnis karena user yang menyumbang revenue dan volume pengiriman pesan yang banyak akan disamakan perlakuannya dengan user yang volume dan revenue nya menjulang tinggi. Alasan selanjutnya juga untuk menghindari ketidakadilan dalam pemberian Treatment (Fairness)

Setelah menjalankan silhoutte analysis, saya mempertimbangkan untuk menggunakan K=3 (paling dekat dengan kriteria ideal). Meskipun secara angka K=2 memiliki skor lebih tinggi, K=3 juga memberikan keseimbangan terbaik antara validitas statistik dan kegunaan bisnis bagi OCA Blast.

Nilai Avg Silhoutte Score untuk K=3 adalah 0.9828. Meskipun nilai ini sedikit di bawah skor K=2, skor ini masih jauh lebih stabil dibandingkan jika kita menaikkan jumlah cluster ke K=4 (0.7877) atau K=5 (0.5870). Angka 0.9828 menunjukkan bahwa dengan membagi

user/klien ke dalam tiga kelompok/cluster, masih bisa dikatakan cukup solid untuk dapat dibedakan satu sama lain. $K=3$ juga memiliki ukuran/proporsi yang seimbang. Nilai negatif (di sebelah kiri angka 0.0) bahkan tidak terlihat, negatifnya bahkan tidak ada, tidak mendominasi. Artinya, tidak terlihat adanya nilai negatif ini membuktikan bahwa tidak ada user yang "salah masuk" ke kelompok yang bukan karakteristiknya.

MENGAPA $K=3$ DIPILIH?

1. Business justification — 3 tier/segmen secara natural TERLIHAT JELAS dari data: ~24.500 tx (3 klien) → ~6.100 tx (5 klien) → ~1.500 tx (12 klien)
2. Silhouette $k=3$ — separation yang cukup baik (di atas 0.9 = very good cluster separation)
3. $k=2$ hanya memisahkan Enterprise vs "semua lainnya" (17 klien dalam 1 kelompok) → tidak actionable untuk strategi bisnis
4. $k>3$ memecah tier yang sudah homogen → over-segmentation tanpa nilai tambah

Interpreting Clustering Result

```
# Create the final cluster model
# Final clustering dengan K=3
km_final = KMeans(n_clusters=3, random_state=42, n_init='auto')
km_final.fit(X_scaled)
```

```
KMeans
KMeans(n_clusters=3, random_state=42)
```

```
# Final clustering dengan K=3
km_final = KMeans(n_clusters=3, random_state=42, n_init=10)
seg['Cluster'] = km_final.fit_predict(X_scaled)
```

```
# Lihat karakteristik tiap cluster
cluster_summary = seg.groupby('Cluster').agg(
    jumlah_klien    = ('user_id', 'count'),
    avg_messages     = ('total_messages', 'mean'),
    avg_revenue      = ('total_revenue', 'mean'),
    avg_msg_per_day  = ('avg_msg_per_day', 'mean'),
    total_revenue    = ('total_revenue', 'sum'),

).round(1)

# Hitung revenue share
cluster_summary['revenue_share_%'] = (
    cluster_summary['total_revenue'] / cluster_summary['total_revenue'].sum() * 100
).round(1)

print("=== Cluster Summary ===")
print(cluster_summary)

print("\n=== User per Cluster ===")
print(seg[['user_id', 'user_name', 'field_of_business', 'total_messages',
    'total_revenue', 'avg_msg_per_day', 'Cluster']]
    .sort_values(['Cluster', 'total_messages'], ascending=[True, False])
    .to_string(index=False))
```

```
=== Cluster Summary ===
   jumlah_klien  avg_messages  avg_revenue  avg_msg_per_day \
Cluster
0              12         1534.4    326725.0             17.0
1               3        24549.7   5177266.7             269.8
2               5         6137.4   1298784.0             67.4

   total_revenue  revenue_share_%
Cluster
0         3920700             15.1
1        15531800             59.9
2         6493920             25.0

=== User per Cluster ===
user_id  user_name  field_of_business  total_messages  total_revenue  avg_msg_per_day  Cluster
user_16  PT Estate Sejahtera      Real Estate             1593         339010             17.5         0
user_18   CV Jalan Mandiri          Travel             1573         344080             17.3         0
user_11  PT Travel Nusantara      Hospitality             1559         340730             17.3         0
```


user_20	CV Hotel Jaya	Hospitality	1554	342240	17.1	0
user_19	CV Niaga Indo	eCommerce	1552	336210	17.2	0
user_13	PT Saham Global	Finance	1530	335960	17.0	0
user_10	PT Mart Jaya	Retail	1525	323700	16.8	0
user_14	CV Hotel Nusantara	Hospitality	1523	305880	16.9	0
user_9	PT Citra Global	Hospitality	1519	317510	16.7	0
user_17	PT Karya Jaya	Education	1510	315770	16.8	0
user_15	PT Edu Mandiri	Education	1488	318840	16.5	0
user_12	CV Mart Indo	Retail	1487	300770	16.3	0
user_3	CV Wisata Indo	Travel	24635	5207360	270.7	1
user_2	PT Dana Nusantara	Finance	24518	5198910	269.4	1
user_1	CV Land Sejahtera	Real Estate	24496	5125530	269.2	1
user_4	PT Konektivitas Indo	Telecommunications	6230	1276070	68.5	2
user_5	PT Karya Sejahtera	Education	6198	1310570	68.1	2
user_7	PT Retail Sejahtera	Retail	6132	1327690	67.4	2
user_8	PT Belajar Nusantara	Education	6117	1284620	67.2	2
user_6	PT Logistik Mandiri	Logistics	6010	1294970	66.0	2

cluster_summary

	jumlah_klien	avg_messages	avg_revenue	avg_msg_per_day	total_revenue	revenue_share_%
Cluster						
0	12	1534.4	326725.0	17.0	3920700	15.1
1	3	24549.7	5177266.7	269.8	15531800	59.9
2	5	6137.4	1298784.0	67.4	6493920	25.0

cluster_summary['revenue_share_%']

	revenue_share_%
Cluster	
0	15.1
1	59.9
2	25.0

dtype: float64

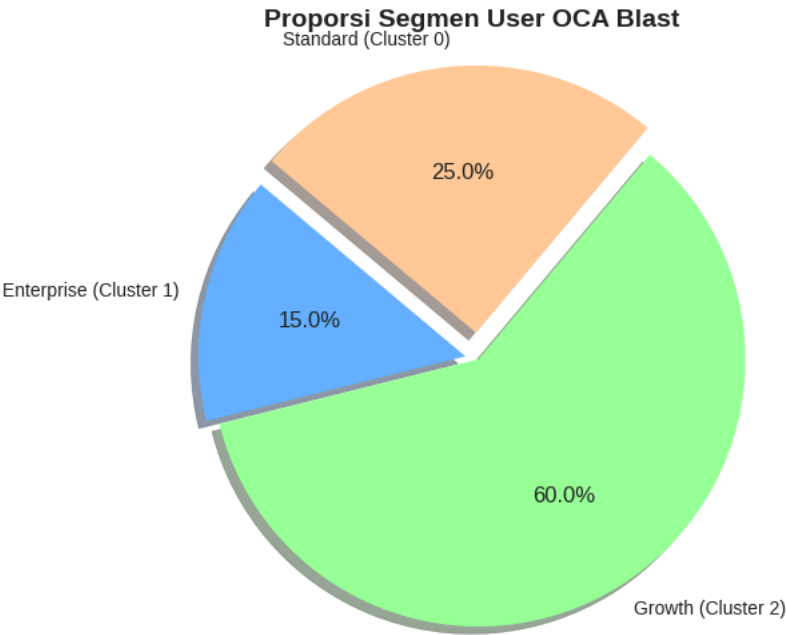
```
import matplotlib.pyplot as plt

# Data berdasarkan jumlah anggota tiap cluster
labels = ['Enterprise (Cluster 1)', 'Growth (Cluster 2)', 'Standard (Cluster 0)']
sizes = [3, 12, 5] # Jumlah anggota
colors = ['#66b3ff', '#99ff99', '#ffcc99'] # Warna untuk estetika
explode = (0.05, 0, 0.1) # Menonjolkan cluster Sultan (paling profit)

# Membuat Pie Chart
plt.figure(figsize=(8, 6))
plt.pie(sizes,
        explode=explode,
        labels=labels,
        colors=colors,
        autopct='%1.1f%%', # Menampilkan persentase
        shadow=True,
        startangle=140)

plt.title('Proporsi Segmen User OCA Blast', fontsize=14, fontweight='bold')
plt.axis('equal') # Memastikan lingkaran berbentuk bulat sempurna

# Menampilkan Chart
plt.show()
```



```
seg[['user_id','user_name','field_of_business','total_messages',  
      'total_revenue','avg_msg_per_day','Cluster']].sort_values(['Cluster','total_messages'], ascending=[True,False])
```

	user_id	user_name	field_of_business	total_messages	total_revenue	avg_msg_per_day	Cluster
7	user_16	PT Estate Sejahtera	Real Estate	1593	339010	17.5	0
9	user_18	CV Jalan Mandiri	Travel	1573	344080	17.3	0
2	user_11	PT Travel Nusantara	Hospitality	1559	340730	17.3	0
12	user_20	CV Hotel Jaya	Hospitality	1554	342240	17.1	0
10	user_19	CV Niaga Indo	eCommerce	1552	336210	17.2	0
4	user_13	PT Saham Global	Finance	1530	335960	17.0	0
1	user_10	PT Mart Jaya	Retail	1525	323700	16.8	0
5	user_14	CV Hotel Nusantara	Hospitality	1523	305880	16.9	0
19	user_9	PT Citra Global	Hospitality	1519	317510	16.7	0
8	user_17	PT Karya Jaya	Education	1510	315770	16.8	0
6	user_15	PT Edu Mandiri	Education	1488	318840	16.5	0
3	user_12	CV Mart Indo	Retail	1487	300770	16.3	0
13	user_3	CV Wisata Indo	Travel	24635	5207360	270.7	1
11	user_2	PT Dana Nusantara	Finance	24518	5198910	269.4	1
0	user_1	CV Land Sejahtera	Real Estate	24496	5125530	269.2	1
14	user_4	PT Konektivitas Indo	Telecommunications	6230	1276070	68.5	2
15	user_5	PT Karya Sejahtera	Education	6198	1310570	68.1	2
17	user_7	PT Retail Sejahtera	Retail	6132	1327690	67.4	2
18	user_8	PT Belajar Nusantara	Education	6117	1284620	67.2	2
16	user_6	PT Logistik Mandiri	Logistics	6010	1294970	66.0	2

```
# Describe each groups of cluster  
profiling = seg.groupby(['Cluster'])
```

```
profiling.describe().round(2)
```

Cluster	total_messages								active_days			...	total_revenue		decline_pct		
	count	mean	min	25%	50%	75%	max	std	count	mean	...	max	std	count	mean	min	
0	12.0	1534.42	1487.0	1516.75	1527.5	1555.25	1593.0	32.60	12.0	90.5	...	344080.0	14919.94	12.0	18.80	12.0	
1	3.0	24549.67	24496.0	24507.00	24518.0	24576.50	24635.0	74.72	3.0	91.0	...	5207360.0	45004.03	3.0	16.89	16.89	
2	5.0	6137.40	6010.0	6117.00	6132.0	6198.00	6230.0	85.03	5.0	91.0	...	1327690.0	20649.20	5.0	17.08	17.08	
3 rows × 136 columns																	

3 rows × 136 columns

Dapat dilihat terdapat perbedaan antara ketiga cluster (cluster 0, cluster 1, dan cluster 2). Perbedaan mencolok dari ketiga cluster/segment terlihat dari total volume pengiriman pesan atau volume transaksinya.

Diketahui bahwa Cluster 0 adalah kelompok dengan jumlah klien yang paling banyak (12 klien) dengan rata-rata total volume per klien nya yang paling rendah (~1.534 transaksi) dan memiliki rata-rata revenue yang dihasilkan juga paling rendah dibandingkan cluster lainnya, yaitu (Rp327K) atau persentase kontribusi terhadap revenue nya hanya 15.1%.

Adapun Cluster 1 (3 klien) adalah kelompok klien dengan total volume pengiriman pesan atau transaksi nya paling tinggi dan rata-rata revenue yang dihasilkan (~24.550 transaksi dan Rp5,2M). Menopang revenue OCA Blast dengan kontribusi terhadap revenue yang dihasilkan hanya dari tiga user/klien mencapai ~60%. User-user di cluster ini paling menguntungkan bagi OCA dan sangat loyal, operasional mereka bergantung penuh kepada platform OCA Blast.

Cluster 2 (5 klien) memiliki rata-rata total volume per klien dan juga rata-rata revenue yang dihasilkan diantara dua cluster lainnya (~6.137 transaksi dan ~Rp1,3M) atau persentase kontribusi terhadap revenue nya sebesar 25%.

Dari sini ditemukan terdapat gap perbedaan yang signifikan antar cluster.

Adapun apabila kita melihat intensitas penggunaan platform (rata-rata pengiriman pesan per harinya) didapatkan bahwa cluster 1 (bisa dibilang top tier cluster) memiliki angka rata-rata pesan per harinya sangat tinggi mencapai 270 transaksi. Intensitas penggunaan nya tinggi, artinya klien-klien di cluster ini mengirimkan pesan dalam jumlah yang sangat banyak per harinya dibandingkan dua cluster lainnya. Cluster 0 (rata rata pengiriman pesan per harinya hanya 17 transaksi/hari) dan Cluster 2 (67 transaksi/hari)

```
# Beri nama cluster
cluster_labels = {1: 'Cluster 1 – Enterprise', 2: 'Cluster 2 – Growth', 0: 'Cluster 3 – Standard'}
seg['Cluster_Label'] = seg['Cluster'].map(cluster_labels)

# Final table – pakai decline_pct & risk_level yang sudah ada
final_kmeans = seg[[
    'user_id', 'user_name', 'field_of_business',
    'Cluster_Label',
    'total_messages', 'avg_msg_per_day', 'active_days',
    'total_revenue',
    'WhatsApp_pct', 'Email_pct', 'SMS_pct', 'Call_pct',
    'decline_pct', 'risk_level'
]].sort_values('total_messages', ascending=False)

print("=== Final Segmentation (K-Means) ===")
print(final_kmeans.to_string(index=False))

print("\n=== Churn Risk per Cluster ===")
print(seg.groupby(['Cluster_Label', 'risk_level'])['user_id'].count().reset_index().to_string(index=False))

OUTPUT_PATH = r"F:\5. Project\Jazmeen\RevouXTelkom\data\cleaned"
final_kmeans.to_csv(f"{OUTPUT_PATH}\\user_segmentation_kmeans.csv", index=False)
print("\nSaved: user_segmentation_kmeans.csv")
```

```
=== Final Segmentation (K-Means) ===
user_id    user_name    field_of_business    Cluster_Label    total_messages    avg_msg_per_day    active_days    total_reve
user_3      CV Wisata Indo    Travel    Cluster 1 – Enterprise    24635    270.7    91    5207
user_2      PT Dana Nusantara    Finance    Cluster 1 – Enterprise    24518    269.4    91    5198
user_1      CV Land Sejahtera    Real Estate    Cluster 1 – Enterprise    24496    269.2    91    5125
user_4      PT Konektivitas Indo    Telecommunications    Cluster 2 – Growth    6230    68.5    91    1276
user_5      PT Karya Sejahtera    Education    Cluster 2 – Growth    6198    68.1    91    1310
user_7      PT Retail Sejahtera    Retail    Cluster 2 – Growth    6132    67.4    91    1327
user_8      PT Belajar Nusantara    Education    Cluster 2 – Growth    6117    67.2    91    1284
user_6      PT Logistik Mandiri    Logistics    Cluster 2 – Growth    6010    66.0    91    1294
user_16     PT Estate Sejahtera    Real Estate    Cluster 3 – Standard    1593    17.5    91    339
user_18     CV Jalan Mandiri    Travel    Cluster 3 – Standard    1573    17.3    91    344
user_11     PT Travel Nusantara    Hospitality    Cluster 3 – Standard    1559    17.3    90    340
user_20     CV Hotel Jaya    Hospitality    Cluster 3 – Standard    1554    17.1    91    342
user_19     CV Niaga Indo    eCommerce    Cluster 3 – Standard    1552    17.2    90    336
```

user_13	PT Saham Global	Finance	Cluster 3 – Standard	1530	17.0	90	335
user_10	PT Mart Jaya	Retail	Cluster 3 – Standard	1525	16.8	91	323
user_14	CV Hotel Nusantara	Hospitality	Cluster 3 – Standard	1523	16.9	90	305
user_9	PT Citra Global	Hospitality	Cluster 3 – Standard	1519	16.7	91	317
user_17	PT Karya Jaya	Education	Cluster 3 – Standard	1510	16.8	90	315
user_15	PT Edu Mandiri	Education	Cluster 3 – Standard	1488	16.5	90	318
user_12	CV Mart Indo	Retail	Cluster 3 – Standard	1487	16.3	91	300

```
=== Churn Risk per Cluster ===
```

Cluster_Label	risk_level	user_id
Cluster 1 – Enterprise	Stable	3
Cluster 2 – Growth	Stable	5
Cluster 3 – Standard	Medium	5
Cluster 3 – Standard	Stable	7

```
Saved: user_segmentation_kmeans.csv
```

Setelah dipilih K=3, 20 klien/user aktif OCA sudah dapat dibagi menjadi tiga segmen, yang terdiri dari:

- Cluster 0 -> Cluster 0 dinamakan Standard/Reguler User. Kelompok ini terdiri dari 12 klien, dimana karakteristik dari cluster atau segmen ini adalah memiliki rata-rata volume transaksi atau volume pesan yang paling rendah diantara dua cluster lainnya (avg volume : 1.534 transaksi dan avg revenue : Rp327K). Cluster ini disebut standard karena klien-klien dari cluster ini menggunakan layanan secara normal, kebutuhan messaging tidak kompleks, volume tidak tinggi, dan kemungkinan hanya untuk operasional dasar
- Cluster 1 -> Cluster 1 dinamakan Enterprise User. Kelompok ini terdiri dari 3 klien (top 3 user berdasarkan volume transaksi dan total revenue yang dihasilkan), dimana karakteristik dari cluster atau segmen ini adalah memiliki rata-rata volume transaksi atau volume pesan tertinggi diantara dua cluster lainnya (avg volume : 24.550 transaksi dan avg revenue : Rp5,2M).
- Cluster 2 -> Cluster 2 dinamakan Growth/Potential User. Kelompok ini terdiri dari 5 klien, dimana karakteristik dari cluster atau segmen ini adalah memiliki rata-rata volume transaksi atau volume pesan yang besarnya diantara dua cluster lainnya, berdekatan dengan cluster 0 (avg volume : 6.137 transaksi dan avg revenue : Rp1,3M).

```
final_kmeans
```

	user_id	user_name	field_of_business	Cluster_Label	total_messages	avg_msg_per_day	active_days	total_revenue	WhatsApp
13	user_3	CV Wisata Indo	Travel	Cluster 1 — Enterprise	24635	270.7	91	5207360	
11	user_2	PT Dana Nusantara	Finance	Cluster 1 — Enterprise	24518	269.4	91	5198910	
0	user_1	CV Land Sejahtera	Real Estate	Cluster 1 — Enterprise	24496	269.2	91	5125530	
14	user_4	PT Konektivitas Indo	Telecommunications	Cluster 2 — Growth	6230	68.5	91	1276070	
15	user_5	PT Karya Sejahtera	Education	Cluster 2 — Growth	6198	68.1	91	1310570	
17	user_7	PT Retail Sejahtera	Retail	Cluster 2 — Growth	6132	67.4	91	1327690	
18	user_8	PT Belajar Nusantara	Education	Cluster 2 — Growth	6117	67.2	91	1284620	
16	user_6	PT Logistik Mandiri	Logistics	Cluster 2 — Growth	6010	66.0	91	1294970	
7	user_16	PT Estate Sejahtera	Real Estate	Cluster 3 — Standard	1593	17.5	91	339010	
9	user_18	CV Jalan Mandiri	Travel	Cluster 3 — Standard	1573	17.3	91	344080	
2	user_11	PT Travel Nusantara	Hospitality	Cluster 3 — Standard	1559	17.3	90	340730	
12	user_20	CV Hotel Jaya	Hospitality	Cluster 3 — Standard	1554	17.1	91	342240	
10	user_19	CV Niaga Indo	eCommerce	Cluster 3 — Standard	1552	17.2	90	336210	
4	user_13	PT Saham Global	Finance	Cluster 3 — Standard	1530	17.0	90	335960	
1	user_10	PT Mart Jaya	Retail	Cluster 3 — Standard	1525	16.8	91	323700	
5	user_14	CV Hotel Nusantara	Hospitality	Cluster 3 — Standard	1523	16.9	90	305880	
19	user_9	PT Citra Global	Hospitality	Cluster 3 — Standard	1519	16.7	91	317510	
8	user_17	PT Karya Jaya	Education	Cluster 3 — Standard	1510	16.8	90	315770	
6	user_15	PT Edu Mandiri	Education	Cluster 3 — Standard	1488	16.5	90	318840	
3	user_12	CV Mart	Retail	Cluster 3 — Standard	1487	16.3	91	300770	

```
print(seg.groupby(['Cluster_Label','risk_level'])['user_id'].count().reset_index().to_string(index=False))
```

Cluster_Label	risk_level	user_id
Cluster 1 – Enterprise	Stable	3
Cluster 2 – Growth	Stable	5
Cluster 3 – Standard	Medium	5
Cluster 3 – Standard	Stable	7

Diketahui Total Revenue OCA Blast selama peiode di data ini adalah Rp25.946.420

❏ Efisiensi Monetisasi per User

```
# -----
# REVENUE EFFICIENCY – rev_per_txn
# -----

# Gabung total_revenue ke user_profile
rev_eff = user_profile[['user_id','total_messages']].merge(user_revenue, on='user_id')

# Hitung rev per transaksi
rev_eff['rev_per_txn'] = (rev_eff['total_revenue'] / rev_eff['total_messages']).round(2)
```

```
# Gabung channel mix
rev_eff = rev_eff.merge(channel_pivot[['user_id','Call_pct','Email_pct','SMS_pct','WhatsApp_pct']], on='user_id')
rev_eff = rev_eff.merge(dfs['Users.csv'][['user_id','user_name','field_of_business']], on='user_id')
rev_eff = rev_eff.merge(seg[['user_id','Cluster_Label']], on='user_id')

# Sort
rev_eff = rev_eff.sort_values('rev_per_txn', ascending=False)

print("=== Revenue Efficiency per User ===")
print(rev_eff[['user_id','user_name','field_of_business','Cluster_Label',
               'total_messages','total_revenue','rev_per_txn',
               'SMS_pct','WhatsApp_pct','Email_pct','Call_pct']].to_string(index=False))

print("\n=== Rev/Tx per Cluster ===")
print(rev_eff.groupby('Cluster_Label')['rev_per_txn'].agg(['mean','min','max']).round(2))

print("\n=== Korelasi Rev/Tx vs Channel Pct ===")
for ch in ['SMS_pct','WhatsApp_pct','Email_pct','Call_pct']:
    corr = rev_eff['rev_per_txn'].corr(rev_eff[ch])
    print(f" {ch:15} → korelasi dengan rev_per_txn: {corr:.3f}")
```

```
=== Revenue Efficiency per User ===
user_id      user_name  field_of_business  Cluster_Label  total_messages  total_revenue  rev_per_txn  SMS_pct  Wha
user_20      CV Hotel Jaya      Hospitality  Cluster 3 – Standard      1554      342240      220.23      16.7
user_13      PT Saham Global      Finance      Cluster 3 – Standard      1530      335960      219.58      16.8
user_18      CV Jalan Mandiri      Travel      Cluster 3 – Standard      1573      344080      218.74      17.2
user_11      PT Travel Nusantara      Hospitality  Cluster 3 – Standard      1559      340730      218.56      16.7
user_19      CV Niaga Indo      eCommerce      Cluster 3 – Standard      1552      336210      216.63      15.9
user_7      PT Retail Sejahtera      Retail      Cluster 2 – Growth      6132      1327690      216.52      16.6
user_6      PT Logistik Mandiri      Logistics      Cluster 2 – Growth      6010      1294970      215.47      16.0
user_15      PT Edu Mandiri      Education      Cluster 3 – Standard      1488      318840      214.27      16.9
user_16      PT Estate Sejahtera      Real Estate      Cluster 3 – Standard      1593      339010      212.81      16.5
user_10      PT Mart Jaya      Retail      Cluster 3 – Standard      1525      323700      212.26      17.0
user_2      PT Dana Nusantara      Finance      Cluster 1 – Enterprise      24518      5198910      212.04      16.3
user_5      PT Karya Sejahtera      Education      Cluster 2 – Growth      6198      1310570      211.45      16.5
user_3      CV Wisata Indo      Travel      Cluster 1 – Enterprise      24635      5207360      211.38      16.4
user_8      PT Belajar Nusantara      Education      Cluster 2 – Growth      6117      1284620      210.01      16.4
user_1      CV Land Sejahtera      Real Estate      Cluster 1 – Enterprise      24496      5125530      209.24      16.2
user_17      PT Karya Jaya      Education      Cluster 3 – Standard      1510      315770      209.12      15.2
user_9      PT Citra Global      Hospitality      Cluster 3 – Standard      1519      317510      209.03      16.1
user_4      PT Konektivitas Indo      Telecommunications      Cluster 2 – Growth      6230      1276070      204.83      15.9
user_12      CV Mart Indo      Retail      Cluster 3 – Standard      1487      300770      202.27      15.7
user_14      CV Hotel Nusantara      Hospitality      Cluster 3 – Standard      1523      305880      200.84      14.8

=== Rev/Tx per Cluster ===
Cluster_Label      mean      min      max
Cluster 1 – Enterprise      210.89      209.24      212.04
Cluster 2 – Growth      211.66      204.83      216.52
Cluster 3 – Standard      212.86      200.84      220.23

=== Korelasi Rev/Tx vs Channel Pct ===
SMS_pct      → korelasi dengan rev_per_txn: 0.729
WhatsApp_pct      → korelasi dengan rev_per_txn: 0.281
Email_pct      → korelasi dengan rev_per_txn: -0.610
Call_pct      → korelasi dengan rev_per_txn: -0.204
```

✓ RCA + Problem Statement + Objective

Problem Statement

"Bagaimana OCA Indonesia dapat mengidentifikasi perilaku dan segmen pengguna aktif berdasarkan data transaksi multi-channel (WhatsApp, SMS, Email, Call) guna mendukung strategi personalisasi layanan dan deteksi churn lebih awal di periode berikutnya?"

Objective

Menganalisis pola perilaku pengguna OCA Blast berdasarkan channel diversity, message volume, usage frequency, dan pola waktu aktif, serta membangun segmentasi pengguna berbasis K-Means clustering yang dapat digunakan sebagai dasar strategi retensi dan personalisasi layanan bagi tim Product dan Business OCA Indonesia.

ROOT CAUSE ANALYSIS (RCA)

Target: Stabilisasi & Pertumbuhan Revenue OCA Blast | — Revenue Performance | ("Siapa" & "Apa" yang Menghasilkan Revenue) | | |
 — Revenue Concentration Risk | | → 3 Enterprise clients = 59.9% revenue | | → Kehilangan 1 Enterprise = hilang ~Rp5.2M sekaligus |
 | | — Volume Gap antar Tier (temuan deskriptif) | → Enterprise 270 tx/hari vs Standard 17 tx/hari (gap 16x) | → Rev/tx hampir identik

(Rp200–220) di semua tier | — Churn & Engagement Risk ("Bagaimana" Klien Berperilaku & Apa Risikonya) | — Pola Penurunan Volume Jan→Mar | → Jan→Feb: semua 20 klien turun (pola universal/seasonal) | → Feb→Mar: mayoritas stabil, 2 klien masih declining | PT Travel Nusantara (–29.3%) & CV Mart Indo (–28.6%) | → Keduanya ada di Standard tier, | sektor Hospitality & Retail | — Karakteristik Pola Waktu Aktif Berbeda per Segmen / Tier → Enterprise: 16–18 WIB → Growth: 21:00 & 00:00 WIB → Standard: 19–23 WIB → Tiap tier memiliki pola komunikasi yang berbeda

✓ Karakteristik Tiap Cluster (Segmen)

1. **Cluster Enterprise:** Cluster/segmen ini hanya terdapat 3 Klien/15% dari total pengguna. Cluster Enterprise sangat mendominasi dari kontribusi terhadap pendapatannya, menjadi backbone bisnis OCA Blast (59.9% dari total revenue). Dilihat dari volume pesannya, rata-rata pesan tertinggi (24.549 pesan/periode atau ~269 pesan/hari). Total volume pesan atau transaksi klien-klien di cluster ini x lebih tinggi dari klien terkecil. Blast adalah inti operasional bisnis mereka (Blast tinggi sekali mencapai ~270/hari) dengan 91 hari aktif, menyumbang revenue terbesar di OCA yakni revenue sharenya mencapai 59.9% total revenue dari tiga klien teratas, diantaranya CV Wisata Indo (Travel), PT Dana Nusantara (Finance), CV Land Sejahtera (Real Estate). Berdasarkan tingkat risiko: Stable (Tingkat penurunan/decline relatif stabil di angka ~15-18%). Akan tetapi, walaupun risk_level nya termasuk kategori stabil, tetap ada risiko, yaitu:

- jika churn → revenue hilang sangat besar
- hanya dari 3 client menyumbang hampir 60% revenue. Ini disebut: Revenue Concentration Risk

Concentration risk sangat tinggi. Zero medium/high churn risk. Ketergantungan operasional penuh pada platform. Klien paling loyal, mereka adalah mesin revenue OCA Blast bukan hanya volume terbesar, tapi juga paling stabil, paling konsisten, dan punya operational dependency tertinggi pada platform. Klien di segmen ini (CV Wisata Indo, PT Dana Nusantara, CV Land Sejahtera) bertindak layaknya Key Accounts atau perusahaan berskala besar yang mengandalkan infrastruktur OCA Blast sebagai inti operasional atau pemasarannya. Kehilangan satu klien saja di segmen ini akan berdampak masif pada cash flow perusahaan (prinsip Pareto 80/20 sangat berlaku di sini).

2. **Cluster Growth:** Cluster/segmen ini secara populasi termasuk Menengah (terdapat 5 klien). Berdasarkan kontribusi terhadap pendapatan OCA Blast, cluster ini persentasenya signifikan (25% dari total revenue). Dilihat dari volume pesannya, termasuk kategori Menengah (rata-rata 6.137 pesan/periode atau ~67 pesan/hari). Berdasarkan tingkat risikonya : Stable, akan tetapi jika dilihat dari avg decline percentage nya mencapai angka 17.1%. Artinya, rata-rata penurunan volume transaksi tiap klien cukup besar. Perlu dipantau lebih ketat klien-kliennya agar tidak ada risiko churn, karena klien-klien di cluster ini berpotensi untuk menjadi loyal customer. Mereka memiliki ruang untuk tumbuh (room to grow) menjadi klien Enterprise jika diberikan dorongan fitur atau kapasitas yang tepat (dilihat juga frekuensi penggunaannya cukup konsisten).

- **Cluster Standard:** Cluster/segmen ini secara populasi termasuk yang terbesar (12 klien / dominasi jumlah pengguna). Berdasarkan kontribusi terhadap pendapatan OCA Blast, cluster ini termasuk yang paling kecil (15.1% dari total revenue). Dilihat dari volume pesannya itu rendah (rata-rata 1.534 pesan/periode atau ~17 pesan/hari). Dilihat dari tingkat risiko: Bervariasi, dengan munculnya status Medium risk pada beberapa klien (5 dari 12 klien termasuk ke risk_level = medium risk, seperti PT Travel Nusantara, CV Niaga Indo, PT Saham Global). Nilai standar deviasi persentase decline paling tinggi (6.08). Channel mix sudah lebih premium. Artinya, proporsi WA/SMS sudah lebih tinggi dari cluster lain secara relatif. Stabil dari segi efisiensi, tapi 5 dari 12 klien masuk ke kategori Medium churn Risk. CV Mart Indo mendekati threshold High Risk (28.6% vs batas 28%) – Urgent dan ada PT Travel Nusantara mendekati High Risk (29.3% mendekati threshold 28%). Dilihat dari avg decline percentage nya mencapai angka 18.8% (TERTINGGI diantara 3 cluster) Perlu intervensi segera. Perlu churn prevention yang tepat untuk klien-klien di segmen ini.

Segmen Enterprise : Cluster Enterprise ini adalah prioritas utama untuk strategi loyalty dan retensi nya. Kehilangan satu user di segmen ini akan sangat berdampak pada revenue. Klien-klien yang ada di segmen ini terdiri dari perusahaan korporat besar (Enterprise) atau agensi marketing yang bergantung penuh pada OCA Blast untuk operasional harian mereka. Mereka menggunakan pendekatan omnichannel (Call, SMS, WA, Email sekaligus). Salah satu upaya yang bisa dilakukan, misalnya kita bisa fokus pada Whatsapp channel karena user/klien di cluster ini condong menggunakan Whatsapp. Fokus pada WhatsApp Business API resmi untuk mencegah blokir nomor. Manfaatkan fitur otomatisasi tingkat lanjut seperti Failover System (pengalihan otomatis dari WhatsApp ke SMS jika pesan gagal terkirim).

Rekomendasi Insentif:

- Fokus Operasional: Ketersediaan server 99.99% (High Availability) dan kecepatan pengiriman tanpa delay.
- Custom API & Volume Discount: Berikan SLA (Service Level Agreement) khusus dan diskon berjenjang (B2B Pricing) untuk mengamankan kontrak jangka panjang.
- Dedicated Account Manager: Berikan jalur VIP khusus untuk customer service (tanpa antrian tiket) atau berikan DEDICATED support